**Szegedi Tudományegyetem**
**Természettudományi és Informatikai Kar**
**Informatikai Intézet**

SZAKDOLGOZAT

# Lightweight kriptográfiai algoritmusok teljesítmény-elemzése erőforrás-korlátozott rendszerekre

Performance Analysis of Lightweight Cryptographic Algorithms for Resource-Constrained Systems

*Készítette:*
**Kárász Barna Kristóf**
Mérnökinformatikus BSc hallgató

*Témavezető:*
**Dr. Békési József**

Szeged
2026

## Feladatkiírás

A hallgató feladata könnyűsúlyú szimmetrikus kriptográfiai algoritmusok teljesítményének vizsgálata erőforrás-korlátozott beágyazott környezetben.

Tekintse át a könnyűsúlyú kriptográfia alapfogalmait, a blokktitkosítók fő szerkezeti típusait és a terület szabványosítási helyzetét. Válasszon ki összehasonlításra néhány jellemző algoritmust, viszonyítási alapként az AES-128-cal együtt.

A kiválasztott algoritmusokat valósítsa meg, illetve illessze be publikált referenciaimplementációjukat egy ARM Cortex-M4 magos mikrovezérlőre, és helyességüket ellenőrizze hivatalos tesztvektorokkal. Dolgozzon ki megismételhető mérési módszert a kulcsgenerálás, a titkosítás és a visszafejtés futási idejének mérésére különböző üzenetméretek és fordítói optimalizációs beállítások mellett, és vizsgálja az implementációk kódméretét és memóriaigényét is.

Az eredményeket értékelje a szakirodalomban közölt biztonsági jellemzőkkel együtt, és fogalmazzon meg ajánlásokat arra, hogy milyen alkalmazási helyzetben melyik algoritmus használata indokolt.

## Tartalmi összefoglaló

**A téma megnevezése:** Lightweight (könnyűsúlyú) kriptográfiai algoritmusok teljesítményelemzése erőforrás-korlátozott rendszerekre.

**A megadott feladat megfogalmazása:** A feladat négy szimmetrikus kriptográfiai algoritmus összehasonlítása volt egy ARM Cortex-M4 magos mikrovezérlőn: az AES-128-é mint viszonyítási alapé, a SPECK 128/128 és a LEA-128 könnyűsúlyú blokktitkosítóé, valamint a NIST SP 800-232 szabvány Ascon-AEAD128 hitelesített titkosítási sémájáé. Meg kellett mérni a futási időt, a kódméretet és a veremigényt, és az eredményeket a biztonsági jellemzőkkel együtt kellett értékelni.

**A megoldási mód:** Mind a négy algoritmusnál nyilvános referenciakódot mértünk (AES: a tervezők T-táblás C-kódja, SPECK: az NSA útmutatója, LEA: a koreai KISA hivatalos kódja, Ascon-AEAD128: a tervezők `ascon-c` könyvtára), egységes mérőkeretben, hivatalos tesztvektorokkal ellenőrizve, egy NUCLEO-L432KC kártyán. A kulcsgenerálás, a titkosítás és a visszafejtés költségét 16 és 1024 bájt közötti üzenetméreteken, hat fordítói optimalizációs szinten mértük, tíz mérési kör mediánjával.

**Alkalmazott eszközök, módszerek:** STM32CubeIDE és GCC, a processzormag DWT ciklusszámlálója, soros porton gyűjtött mérési naplók, Python-alapú kiértékelés (matplotlib), a kódméret és a veremigény meghatározása a linker térképfájljából és a GCC hívási gráfjából, a generált gépi kód elemzése, a biztonsági jellemzők szakirodalmi áttekintése.

**Elért eredmények:** `-O3` optimalizáció mellett a SPECK és a LEA fajlagos költsége gyakorlatilag azonos volt, és titkosításkor 1,8-szer, visszafejtéskor 2,0-szer kisebb, mint az AES T-táblás referenciakódjáé. Az Ascon-AEAD128 1024 bájton a hitelesítéssel együtt is csak 4 %-kal volt drágább a csak titkosító AES-nél, egy becsült AES-CCM-nél pedig már 16 bájton olcsóbb. A LEA és az Ascon-AEAD128 kibontott kódja nem fér el az 1 KB-os utasítás-gyorsítótárban, ezért futási idejüket nagyrészt a flash-memória olvasása szabja meg. Egy elterjedt, kis kódméretre írt AES-könyvtár (`tiny-AES-c`) 2,5–7,2-szer lassabbnak bizonyult a referenciakódnál, vagyis az implementáció megválasztása az algoritmusok közötti különbséget is felülírhatja. A mérések alapján alkalmazási ajánlásokat fogalmaztunk meg.

**Kulcsszavak:** könnyűsúlyú kriptográfia, blokktitkosító, hitelesített titkosítás, Ascon, Cortex-M4, teljesítménymérés

# Tartalomjegyzék


**1. Bevezetés** **6**

**2. Háttérismeretek** **9**

2.1. Szimmetrikus kriptográfia alapjai . . . . . 9

2.1.1. Blokktitkosítók . . . . . 9

2.1.2. Üzemmódok . . . . . 11

2.1.3. A DES és az AES rövid áttekintése . . . . . 11

2.2. Hashfüggvények és üzenethitelesítés . . . . . 12

2.2.1. Kriptográfiai hashfüggvények . . . . . 12

2.2.2. MAC – üzenethitelesítő kód . . . . . 13

2.2.3. AEAD – hitelesített titkosítás kapcsolódó adatokkal . . . . . 13

2.3. Erőforrás-korlátozott rendszerek jellemzői . . . . . 13

2.3.1. Mikrovezérlők és erőforrásaik . . . . . 13

2.4. A lightweight kriptográfia motivációja . . . . . 14

2.4.1. A hagyományos szabványok korlátai . . . . . 14

2.4.2. A NIST Lightweight Cryptography szabványosítás . . . . . 14

**3. Vizsgált algoritmusok bemutatása** **16**

3.1. AES-128 . . . . . 16

3.1.1. Felépítés és működés . . . . . 16

3.1.2. Kulcsgenerálás . . . . . 17

3.1.3. Az AES-128 mint viszonyítási alap . . . . . 17

3.2. SPECK 128/128 . . . . . 17

3.2.1. Körfüggvény . . . . . 18

3.2.2. Kulcsgenerálás . . . . . 18

3.2.3. Szoftveres megvalósítás . . . . . 18

3.2.4. Bizalmi kérdések . . . . . 19

3.3. LEA-128 . . . . . 19

3.3.1. Körfüggvény . . . . . 19

3.3.2. A 32 bites szóméret . . . . . 20

3.3.3. Kulcsgenerálás . . . . . 20

3.4. Ascon-AEAD128 . . . . . 20

3.4.1. Szivacs- és duplexkonstrukció . . . . . 20

3.4.2. Az Ascon-permutáció . . . . . 21

3.4.3. Az AEAD működése . . . . . 21

3.4.4. A vizsgált változat . . . . . 22

3.5. Az algoritmusok összevetése . . . . . 22

**4. Implementáció és mérési módszer** **24**

4.1. A mérési platform . . . . . 24

4.2. A forráskód szerkezete . . . . . 25

4.3. Az algoritmusok megvalósítása . . . . . 25

4.3.1. AES-128 . . . . . 25

4.3.2. SPECK 128/128 . . . . . 26

4.3.3. LEA-128 . . . . . 26

4.3.4. Ascon-AEAD128 . . . . . 27

4.4. A mérési keretrendszer . . . . . 27

4.4.1. Önellenőrzés és bemenetek . . . 27
4.4.2. Ciklusmérés a DWT-egységgel . . . 28
4.4.3. Ismétlések, statisztika és kiírás . . . 29
4.5. Az üzemmódok megválasztása . . . 29
4.6. Fordítási beállítások . . . 29
**5. Teljesítménymérések és eredmények 31**
5.1. A kulcsgenerálás költsége . . . 31
5.2. Titkosítás és visszafejtés . . . 32
5.3. Skálázódás az üzenetmérettel . . . 33
5.4. Feldolgozási sebesség . . . 35
5.5. Kódméret és memóriaigény . . . 36
5.6. A fordítói optimalizáció hatása . . . 37
5.6.1. Optimalizálás nélkül és optimalizálva . . . 37
5.6.2. `-O2`, `-O3` és `-Ofast` . . . 39
5.6.3. `-Os/-Oz` . . . 39
5.6.4. A LEA és az utasítás-gyorsítótár . . . 39
5.7. Összevetés a `tiny-AES-c` könyvtárral . . . 40
5.8. Összevetés a szakirodalmi eredményekkel . . . 41
5.9. Az eredmények összegzése . . . 42
**6. Biztonsági elemzés és összehasonlító értékelés 43**
6.1. Alapfogalmak . . . 43
6.1.1. Támadási modellek . . . 43
6.1.2. Biztonsági szint és biztonsági tartalék . . . 43
6.2. A főbb támadástípusok . . . 44
6.2.1. Differenciális kriptoanalízis . . . 44
6.2.2. Lineáris kriptoanalízis . . . 44
6.2.3. Algebrai és rotációs támadások . . . 44
6.2.4. Oldalcsatorna-támadások . . . 45
6.2.5. Születésnapi korlát . . . 45
6.3. Az algoritmusok biztonsági jellemzői . . . 45
6.3.1. AES-128 . . . 45
6.3.2. SPECK 128/128 . . . 46
6.3.3. LEA-128 . . . 46
6.3.4. Ascon-AEAD128 . . . 46
6.4. Összehasonlító értékelés . . . 46
**7. Összefoglalás és továbbfejlesztési lehetőségek 48**
7.1. Felhasznált módszerek és eszközök . . . 48
7.2. A saját munka . . . 48
7.3. Az eredmények tézisszerű összefoglalása . . . 49
7.4. Alkalmazási ajánlások . . . 50
7.5. Korlátok . . . 51
7.6. Továbbfejlesztési lehetőségek . . . 52
**Irodalomjegyzék 53**
**Nyilatkozat 56**
**Köszönetnyilvánítás 57**
**Mellékletek 58**

# 1. fejezet

# Bevezetés

Az elmúlt évtizedben a hálózatba kapcsolt eszközök az ipar, a közlekedés és a háztartások mindennapi részévé váltak. Az *Ipar 4.0* keretében a gyártósorok, a logisztikai láncok és az épületgépészeti rendszerek egyre inkább egymással kommunikáló eszközökre épülnek [25]. A dolgok internete (Internet of Things, IoT) ma már nem csupán kutatási téma: okos termosztátok, viselhető egészségügyi eszközök, mezőgazdasági szenzorok és közlekedésirányító rendszerek alkotják.

Az Ericsson jelentése szerint 2025 végén világszerte 22,3 milliárd IoT-kapcsolat volt, és ez a szám 2031-re megközelítheti a 47,1 milliárdot [24]. Ezeknek az eszközöknek jelentős része nem számítógép vagy okostelefon, hanem mikrovezérlő, szenzor vagy RFID-címke, amely kevés memóriával, alacsony órajellel és szűkös, gyakran elemről biztosított energiaellátással működik.

Ezek az eszközök sokszor érzékeny adatokat kezelnek. Egy gyártósoron a meghamisított szenzorérték vagy vezérlőparancs anyagi és fizikai kárt okozhat, egy közlekedési rendszerben a módosított adat balesethez vezethet, egy egészségügyi eszköznél pedig a kiszivárgó betegadat súlyos adatvédelmi következményekkel jár. A kommunikáció titkosítása és hitelesítése ezért ezeken az eszközökön is alapkövetelmény [23].

A széles körben használt kriptográfiai eljárásokat nem elsősorban ilyen eszközökre tervezték. Az aszimmetrikus algoritmusok (pl. RSA) egy kis mikrovezérlőn lassúak és sok energiát fogyasztanak, de még a szimmetrikus AES is érezhető részt foglalhat el a legszűkebb eszközök memóriájából és számítási idejéből. Erre a problémára kínál megoldást a *könnyűsúlyú* (lightweight) *kriptográfia*, amely kevés erőforrással is az adott alkalmazáshoz elegendő biztonságot igyekszik nyújtani [6].

A terület jelentőségét jól mutatja, hogy a NIST (National Institute of Standards and Technology) 2018-ban külön szabványosítási eljárást indított a könnyűsúlyú kriptográfiára, és 2023-ban a grazi kutatók által tervezett Ascont választotta ki szabványnak [10].

A dolgozatban négy, eltérő felépítésű kriptográfiai algoritmust hasonlítunk össze. Célunk annak megmérése, hogy az egyes algoritmusok mekkora futásidejű költséggel

járnak egy erőforrás-korlátozott eszközön. A méréseket egy ARM Cortex-M4 magos STM32L432KC mikrovezérlőn végeztük: a kulcsgenerálás, a titkosítás és a visszafejtés ciklusszámát mértük, ebből pedig a fajlagos költséget és a feldolgozási sebességet határoztuk meg, hat fordítói optimalizációs szinten. A vizsgált algoritmusok:

1. **AES-128** – az iparban legelterjedtebb szimmetrikus blokktitkosító, amely az összehasonlításban viszonyítási alapként szolgál;
2. **SPECK 128/128** – az NSA által tervezett, szoftveres futásra optimalizált, ARX-alapú könnyűsúlyú blokktitkosító;
3. **LEA-128** – a dél-koreai NSRI által kidolgozott könnyűsúlyú blokktitkosító, amely az ISO/IEC 29192-2:2019 szabvány része;
4. **Ascon-AEAD128** – a NIST által 2023-ban kiválasztott Ascon család szivacskonstrukcióra épülő hitelesített titkosítási sémája, a NIST SP 800-232 szabvány szerinti változatban.

Az algoritmusokat úgy választottuk ki, hogy mindegyik más tervezési elvet képviseljen (SPN, két eltérő szóméretű ARX-szerkezet, illetve szivacskonstrukció), ugyanakkor összevethetők maradjanak: mind a négy 128 bites kulcsot használ, a blokktitkosítók 128 bites blokkot, és mindegyik jól megvalósítható szoftverben. Így a mért különbségeket nemcsak közölni, hanem az algoritmusok felépítésével magyarázni is tudjuk.

A dolgozat a következő kérdésekre keres választ:

K1. Mekkora a négy algoritmus futási ideje, fajlagos költsége, kódmérete és veremigénye egy Cortex-M4 magos mikrovezérlőn, és hogyan változik a költség az üzenet hosszával?

K2. Hogyan viszonyul a hitelesítést is nyújtó Ascon-AEAD128 költsége a csak titkosító blokktitkosítókéhoz, és milyen üzenetmérettől kedvezőbb egy hitelesített AES-alapú megoldásnál?

K3. Mennyire függnek az eredmények a fordítási beállítástól, és mi magyarázza az eltéréseket?

K4. Jár-e a könnyűsúlyú algoritmusok teljesítményelőnye biztonsági hátránnyal a publikált kriptoanalízis alapján?

Az eredményeket a 7.3. alfejezet ezekhez a kérdésekhez rendelve foglalja össze.

A dolgozat felépítése a következő. A 2. fejezet a szimmetrikus kriptográfia alapjait, a hashfüggvényeket és az üzenethitelesítést, az erőforrás-korlátozott rendszerek jellemzőit, valamint a könnyűsúlyú kriptográfia megjelenésének okait mutatja be. A 3. fejezet a négy

kiválasztott algoritmust ismerteti és veti össze. A 4. fejezet a szoftveres megvalósítást és a mérési módszert, az 5. fejezet a mérési eredményeket tárgyalja. A 6. fejezet a szakirodalom alapján összeveti az algoritmusok biztonsági jellemzőit, végül a 7. fejezet összegzi az eredményeket, alkalmazási ajánlásokat fogalmaz meg, és továbbfejlesztési lehetőségeket vázol fel.

# 2. fejezet

# Háttérismeretek

Ez a fejezet azokat az alapokat foglalja össze, amelyekre a dolgozat további része épül: a szimmetrikus kriptográfia fő fogalmait, a hashfüggvényeket és az üzenethitelesítést, az erőforrás-korlátozott eszközök jellemzőit, valamint azokat a gyakorlati okokat, amelyek miatt a könnyűsúlyú kriptográfia önálló kutatási területté vált.

## 2.1. Szimmetrikus kriptográfia alapjai

A *szimmetrikus kriptográfiában* a titkosítás és a visszafejtés *ugyanazzal a titkos kulccsal* történik. Formálisan: adott egy $E$ titkosító és egy $D$ visszafejtő függvény, amelyekre tetszőleges $P$ nyílt szöveg és $K$ kulcs esetén $D(K, E(K, P)) = P$ [1]. Az aszimmetrikus (nyilvános kulcsú) eljárásoknál lényegesen kevesebb számítást igényelnek, ezért erőforrás-korlátozott eszközökön elsősorban ezeket használják az adatok titkosítására.

### 2.1.1. Blokktitkosítók

A szimmetrikus titkosítás leggyakoribb formája a *blokktitkosító* (block cipher). Ez a bemenetet rögzített méretű, jellemzően 64 vagy 128 bites egységekre, úgynevezett *blokkokra* bontja, és mindegyiket a kulcs függvényében egy azonos méretű titkosított blokká alakítja [2]. A blokktitkosító több *körből* (round) áll. Egy-egy kör önmagában egyszerű műveleteket végez, de kellő számú ismétlés után a kimenet már nem mutat kihasználható összefüggést a bemenettel és a kulccsal. A tervezés két alapelvét Claude Shannon fogalmazta meg 1949-ben [3]:

- **Konfúzió** (confusion): a titkosított szöveg és a kulcs közötti kapcsolatnak a lehető legbonyolultabbnak kell lennie. Ezt nemlineáris műveletek biztosítják, például helyettesítő táblák (S-boxok) vagy moduláris összeadás.
- **Diffúzió** (diffusion): a nyílt szöveg egyetlen bitjének megváltozása a titkosított szöveg minél több bitjére hasson ki. Ezt permutációk, bitforgatások vagy lineáris

transzformációk (pl. véges test feletti mátrixszorzás) érik el.

A következőkben a blokktitkosítók három gyakori felépítését mutatjuk be. A dolgozatban vizsgált blokktitkosítók mindegyike ezek valamelyikére épül.

### SPN – helyettesítő-permutációs hálózat

A *Substitution-Permutation Network* (SPN) felépítésű blokktitkosítók minden körben három réteget alkalmaznak. A *helyettesítő réteg* (substitution layer) a blokkot kisebb, általában 4 vagy 8 bites részekre bontja, és mindegyiket egy rögzített nemlineáris leképezésen, az *S-boxon* vezeti át. Az S-box felel elsősorban a konfúzióért.

A második, *diffúziós réteg* szétteríti az S-boxok kimenetét a blokkon, így a következő körben minden S-box több előző S-box eredményét kapja meg. Egyszerűbb SPN-eknél ez a bitek átrendezése (permutációja), az AES-nél viszont egy lineáris transzformáció (ShiftRows és MixColumns). Néhány kör után így a nyílt szöveg egyetlen bitjének változása a teljes kimeneti blokkot érinti. Végül a *körkulcs hozzáadása* (round key addition) során egy XOR művelet keveri az állapotba a fő kulcsból származtatott körkulcsot. Az SPN legismertebb képviselője az AES [5].

### Feistel-hálózat

A Horst Feistelről elnevezett szerkezet a blokkot két egyenlő félre, $L$-re (bal) és $R$-re (jobb) bontja. Minden körben az $R$ félre egy $F$ körfüggvényt alkalmaz az aktuális körkulccsal, az eredményt XOR-ral hozzáadja az $L$ félhez, majd a két felet felcseréli:

$$\begin{aligned} L_{i+1} &= R_i, \\ R_{i+1} &= L_i \oplus F(R_i, K_i). \end{aligned} \tag{2.1}$$

A Feistel-hálózat nagy előnye, hogy a visszafejtés ugyanazzal a szerkezettel történik, mint a titkosítás, csak a körkulcsokat kell fordított sorrendben alkalmazni. Ezért az $F$ függvénynek nem kell invertálhatónak lennie [2]. A szerkezet legismertebb példája a DES (Data Encryption Standard), amelynek 16 körös Feistel-hálózata 1977-től évtizedekig meghatározó szabvány volt. 56 bites kulcsa azonban ma már nem véd a kulcstér végigpróbálása (brute force) ellen.

### ARX-szerkezet

Az *Add-Rotate-XOR* (ARX) megközelítés három egyszerű műveletre épül [12]:

- **Add** – moduláris összeadás ($\bmod 2^n$): az átvitelek terjedése miatt ez a nemlineáris elem, amely S-box nélkül biztosítja a konfúziót;

– **Rotate** – bitforgatás (ciklikus eltolás): a szón belüli diffúziót szolgálja;

– **XOR** – kizáró vagy: a körkulcs bekeverésére és az ágak összekeverésére szolgál.

Az ARX fő előnye az egyszerűség. Ha a szóméret megegyezik a processzor regiszterszélességével, mindhárom művelet egyetlen gépi utasítás, és nincs szükség memóriában tárolt helyettesítő táblára. Mivel a műveletek végrehajtási ideje nem függ a feldolgozott adattól, az ARX-algoritmusok könnyen megvalósíthatók *konstans idejű* (constant-time) kóddal, ami fontos védelem az időzítéses oldalcsatorna-támadások (timing attacks) ellen. A dolgozatban vizsgált SPECK és LEA egyaránt ARX-alapú, de eltérő szerkezetben használja ezeket a műveleteket.

### 2.1.2. Üzemmódok

A blokktitkosító egyszerre csak egy rögzített méretű blokkot dolgoz fel, a valódi üzenetek hossza viszont tetszőleges. Azt, hogy az egymást követő blokkokat hogyan titkosítjuk és hogyan kapcsoljuk össze, az *üzemmód* (mode of operation) határozza meg. A legfontosabbak [1]:

– **ECB** (Electronic Codebook): minden blokkot egymástól függetlenül, ugyanazzal a kulccsal titkosítunk. Egyszerű és párhuzamosítható, de az azonos nyíltszöveg-blokkokból mindig azonos titkosított blokkok lesznek, így az adat mintázatai (például egy kép körvonalai) a titkosított szövegben is látszanak. Ezért önmagában nem ajánlott.

– **CBC** (Cipher Block Chaining): titkosítás előtt minden nyíltszöveg-blokkot XOR-ral összekapcsolunk az *előző* titkosított blokkal; az első blokkhoz egy véletlenszerű inicializációs vektort (IV) használunk. A láncolás miatt azonos bemeneti blokkokból különböző kimenet keletkezik. Hátránya, hogy a titkosítás nem párhuzamosítható.

– **CTR** (Counter Mode): a blokktitkosító egy minden blokknál más számlálóértéket titkosít, és az így kapott *kulcsfolyamot* (keystream) XOR-ral adjuk a nyílt szöveghez. A számlálóértékek előre ismertek, ezért a titkosítás és a visszafejtés is párhuzamosítható, a visszafejtéshez pedig elég a titkosító függvény. Lényeges feltétel, hogy ugyanazzal a kulccsal egy számlálóérték soha ne ismétlődjön.

### 2.1.3. A DES és az AES rövid áttekintése

A *Data Encryption Standard* (DES) az 1977-es szabványosítás után évtizedekig a legszélesebb körben használt szimmetrikus algoritmus volt. Feistel-hálózata 16 körből áll, a

nemlineáris lépést 8 darab, 6 bites bemenetet 4 bites kimenetre képező S-box végzi. A szabvány bukását az 56 bites kulcs okozta: 1998-ban az Electronic Frontier Foundation (EFF) erre a célra épített gépe 56 óra alatt találta meg a kulcsot, 1999-ben pedig a distributed.net önkéntes hálózatával közösen alig több mint 22 óra alatt [2]. A Triple-DES (3DES) ideiglenesen megoldotta a kulcsméret problémáját, de lassú volt, és 64 bites blokkmérete is korlátot jelentett, ezért új szabványra volt szükség.

Az *Advanced Encryption Standard* (AES) 2001-ben váltotta fel a DES-t. A NIST nyílt pályázatát a belga Joan Daemen és Vincent Rijmen Rijndael nevű algoritmusa nyerte meg [4]. Az AES SPN-szerkezetű, 128 bites blokkokkal dolgozik, kulcsa 128, 192 vagy 256 bites, amelyekhez rendre 10, 12 és 14 kör tartozik. Ma az AES a szimmetrikus kriptográfia etalonja: számos processzor hardveresen gyorsítja (pl. az Intel AES-NI utasításkészlettel), és a teljes algoritmus ellen nem ismert gyakorlatban kivitelezhető támadás. Ezért szolgál az AES-128 a dolgozatban viszonyítási alapként: azt vizsgáljuk, mennyi erőforrást takaríthatnak meg hozzá képest a könnyűsúlyú algoritmusok, és ennek milyen biztonsági ára van.

## 2.2. Hashfüggvények és üzenethitelesítés

A biztonságos kommunikációhoz a titkosság mellett *integritás* (az adat sértetlensége) és *hitelesség* (a feladó igazolása) is szükséges [1]. Ezeket kriptográfiai hashfüggvényekkel, üzenethitelesítő kódokkal (MAC) és hitelesített titkosítási sémákkal (AEAD) lehet biztosítani.

### 2.2.1. Kriptográfiai hashfüggvények

A hashfüggvény tetszőleges hosszúságú bemenetből rögzített méretű kimenetet, *lenyomatot* (digest) állít elő. Biztonsági célra akkor alkalmas, ha teljesíti a következő három feltételt [2]:

1. **Egyirányúság** (preimage resistance): a lenyomatból gyakorlatilag ne lehessen olyan bemenetet találni, amely ezt a lenyomatot adja.

2. **Második ősképpel szembeni ellenállás** (second preimage resistance): egy adott bemenethez ne lehessen olyan másik bemenetet találni, amelynek ugyanaz a lenyomata.

3. **Ütközésállóság** (collision resistance): ne lehessen hatékonyan találni két tetszőleges, különböző bemenetet azonos lenyomattal.

Ma a SHA-2 és a SHA-3 család a szabványos választás; az MD5 és a SHA-1 a megtalált ütközések miatt biztonsági célra már nem használható.

### 2.2.2. MAC – üzenethitelesítő kód

A MAC (Message Authentication Code) előállításához egy közös titkos kulcs is kell, ezért az integritás mellett a hitelességet is biztosítja: érvényes MAC-ot csak az tud előállítani, aki ismeri a kulcsot. A két legelterjedtebb változat a hashfüggvényre épülő HMAC [1] és a blokktitkosítót láncolva használó CBC-MAC.

### 2.2.3. AEAD – hitelesített titkosítás kapcsolódó adatokkal

A gyakorlat azt mutatta, hogy a titkosítást és a hitelesítést célszerű egyetlen, együtt tervezett eljárásban elvégezni. Ezt valósítják meg az AEAD (Authenticated Encryption with Associated Data) sémák [9]. Bemenetük a titkos kulcs, egy csak egyszer használható érték (nonce), a nyílt szöveg és a *kapcsolódó adat* (associated data, AD). Ez utóbbi, például egy hálózati csomag fejléce, titkosítatlan marad, hogy a hálózati eszközök olvashassák, de a séma hitelesíti, így módosítása kiderül.

A kimenet a titkosított szöveg és egy *hitelesítő címke* (tag), amely a titkosított szöveg és a kapcsolódó adat sértetlenségét igazolja. Az AEAD sémák egyik legfontosabb feltétele, hogy ugyanazzal a kulccsal egy nonce-ot ne használjunk kétszer, mert ez a legtöbb sémánál súlyosan sérti a biztonságot. A dolgozat szempontjából azért fontos az AEAD, mert az Ascon a többi vizsgált algoritmussal ellentétben nem blokktitkosító, hanem eleve ilyen sémának készült.

## 2.3. Erőforrás-korlátozott rendszerek jellemzői

A könnyűsúlyú kriptográfia célpontjai jellemzően kis teljesítményű, kevés memóriával és szűkös energiával rendelkező eszközök.

### 2.3.1. Mikrovezérlők és erőforrásaik

Az IoT-eszközökben használt processzorok szóméretük szerint három fő csoportba sorolhatók:

- **8 bites** (pl. ATmega): néhány MHz-es órajel, néhány kilobájt RAM. Egyszerű szenzorokban és beágyazott vezérlőkben gyakoriak.
- **16 bites** (pl. MSP430): nagyon alacsony fogyasztásra tervezett platformok, amelyeket gyakran használnak vezeték nélküli szenzorhálózatokban (WSN) [6].
- **32 bites** (pl. ARM Cortex-M, ESP32): ipari átjárókban és összetettebb szenzorrendszerekben használt eszközök, akár megabájtos flash-memóriával, de még így is nagyságrendekkel kisebb kapacitással, mint egy asztali számítógép.

Ezek a korlátok közvetlenül meghatározzák, milyen kriptográfiai megoldás futtatható az eszközön. A programkód és a konstansok a flash-memóriában, a változók és a körkulcsok a jellemzően sokkal kisebb RAM-ban kapnak helyet; egy RFID-címkén például mindössze 300–500 bájt RAM állhat rendelkezésre [8]. Sok kis processzorból hiányzik a hardveres szorzó vagy a kriptográfiai gyorsító, így minden műveletet egyszerű utasításokból kell felépíteni. Elemes eszközöknél minden elvégzett óraciklus energiát fogyaszt, ezért a kriptográfiai műveletek költsége is befolyásolja, meddig működik az eszköz egy töltéssel vagy elemcserével [6].

Ilyen eszközök működnek többek között az ipari rendszerekben (például PLC-k és SCADA-rendszerek környezetében) [25], az okosváros-infrastruktúrában és az egészségügyi eszközökben, ahol egyszerre van szükség erős adatvédelemre és takarékos működésre. Ezeken a területeken válik igazán fontossá a könnyűsúlyú kriptográfia.

## 2.4. A lightweight kriptográfia motivációja

### 2.4.1. A hagyományos szabványok korlátai

Az AES-t kifejezetten úgy tervezték, hogy sokféle platformon, a 8 bites smart cardoktól a szerverekig hatékonyan fusson [5]. A legszűkebb eszközökön azonban így is jelentős a költsége: a 256 bájtos S-box tábla a néhány kilobájtos memóriából érezhető részt foglal el, a véges test feletti MixColumns lépés pedig 8 bites processzoron sok utasítást igényel. A gyors 32 bites szoftveres megvalósítások ezt a számítást több kilobájtos előre kiszámolt táblákkal váltják ki (3.1.1. alfejezet). A legkisebb ismert hardveres megvalósítása is kb. 2400 kapuegyenértéknyi (gate equivalent, GE) területet foglal [45], ami a legolcsóbb RFID-chipeken túl sok [6].

A könnyűsúlyú kriptográfia ezért a szűkös erőforrások és az elvárt biztonság közötti egyensúlyt keresi. Egy IoT-eszköz általában nem igényel 256 bites biztonsági szintet; a 128 bites szint a jelenlegi ismeretek szerint az eszköz teljes élettartamára elegendő védelmet nyújt [7]. A cél tehát az, hogy ez a biztonsági szint a lehető legkevesebb memóriával és energiával legyen elérhető, például kisebb S-boxokkal vagy egyszerű ARX-műveletekkel.

### 2.4.2. A NIST Lightweight Cryptography szabványosítás

A NIST 2018-ban hirdetett nyílt pályázatot erőforrás-korlátozott eszközökre szánt hitelesített titkosítási (AEAD) sémák szabványosítására [10]. A jelöltek kulcsa legalább 128 bites lehetett, és a kriptoanalitikai támadásoknak legalább $2^{112}$ műveletet kellett igényelniük, a meglévő NIST-szabványoknál lényegesen jobb hatékonyság mellett.

A beérkezett 57 pályázatból 56 jutott az első fordulóba, 2021-ben tíz döntős maradt, és

2023 februárjában a NIST az *Ascont* választotta ki. A döntésben szerepet játszott, hogy az Ascon egyetlen, 320 bites állapoton dolgozó permutációra épül, amelyet szivacs- (duplex-) konstrukcióban használ, így egyszerűen és hatékonyan megvalósítható hardverben és szoftverben is [9]. A végleges szabvány NIST SP 800-232 néven 2025 augusztusában jelent meg [11].

# 3. fejezet

# Vizsgált algoritmusok bemutatása

Ez a fejezet a négy vizsgált algoritmus felépítését, kulcsgenerálását és azokat a tulajdonságait mutatja be, amelyekre a mérési eredmények értelmezésénél támaszkodunk.

## 3.1. AES-128

Az AES eredeti neve *Rijndael*, amely a két belga tervező, Joan Daemen és Vincent Rijmen nevéből állt össze. A NIST 1997-ben kiírt pályázatára 15 jelölt érkezett; a Rijndael a biztonsága, a hatékonysága és a sokféle platformon való jó megvalósíthatósága miatt nyert, és 2001-ben FIPS 197 néven szabvány lett [4, 5].

### 3.1.1. Felépítés és működés

Az AES-128 SPN-szerkezetű blokktitkosító: 128 bites blokkokat dolgoz fel 128 bites kulccsal, 10 körben. A belső állapot (*State*) egy $4 \times 4$-es bájtmátrix. Egy kör négy lépésből áll:

1. **SubBytes** – minden bájtot egy 256 elemű S-boxon vezet át. Az S-box a $GF(2^8)$ feletti multiplikatív inverzen és egy affin transzformáción alapul, ez adja a konfúziót.
2. **ShiftRows** – a mátrix *i*-edik sorát *i* pozícióval balra forgatja, így egy oszlop bájtjai a következő lépésben különböző oszlopokba kerülnek.
3. **MixColumns** – minden oszlopot egy rögzített mátrixszal szoroz $GF(2^8)$ felett, így egy bemeneti bájt az oszlop mind a négy kimeneti bájtjára hat. Az utolsó körből ez a lépés kimarad.
4. **AddRoundKey** – XOR-ral hozzáadja az állapothoz az aktuális körkulcsot.

A visszafejtés ezek inverzeit alkalmazza fordított sorrendben. A titkosító MixColumns együtthatói kicsik (1, 2, 3), ezért a szorzás egy-egy kettővel való szorzásra (*xtime*) és

XOR-ra vezethető vissza. Az inverz MixColumns együtthatói (9, 11, 13, 14) viszont nagyobbak, így bájtonként számolva lényegesen több műveletet igényel.

32 bites processzoron ennél gyorsabb, táblás megvalósítás is lehetséges [5]. A SubBytes, a ShiftRows és a MixColumns együttes hatása egy oszlopra négy előre kiszámolt, 256 elemű, 32 bites táblával (*T-táblával*) írható le, így egy kör oszloponként négy táblaolvasásra és négy XOR-ra egyszerűsödik. A visszafejtéshez ugyanígy négy inverz tábla készíthető. Ehhez a FIPS 197 *ekvivalens inverz titkosítását* (equivalent inverse cipher) használják: a lépések sorrendje felcserélhető, ha a visszafejtő körkulcsokra is alkalmazzuk az inverz MixColumns-t [4]. Ez a kulcsgenerálásnál egyszeri többletmunka, cserébe a visszafejtés pontosan olyan felépítésű és költségű, mint a titkosítás. Az ár a memória: az utolsó körhöz használt egy-egy további táblával együtt tíz 1 KB-os táblára van szükség. A táblaolvasások címe ráadásul a kulcstól és az adattól függ, ezért ez a megoldás nem konstans idejű (6.2. alfejezet).

### 3.1.2. Kulcsgenerálás

Az AES-128 kulcsgenerálása a 128 bites fő kulcsból 11 darab 128 bites körkulcsot (összesen 176 bájtot) állít elő; az első körkulcs maga a fő kulcs. Egy új körkulcs első oszlopához az előző körkulcs utolsó oszlopát egy bájttal elforgatjuk, bájtjait átvezetjük az S-boxon, hozzáadjuk a körkonstanst (Rcon), végül XOR-ral összekeverjük az előző körkulcs első oszlopával. A további oszlopok az előző oszlop és az előző körkulcs megfelelő oszlopának XOR-jaként adódnak [5].

### 3.1.3. Az AES-128 mint viszonyítási alap

Az AES-128 a dolgozatban nem könnyűsúlyú algoritmusként, hanem viszonyítási alapként szerepel, mert a legtöbb mai protokoll (TLS, VPN, banki rendszerek) erre épül. Korlátozott eszközökön kétféleképpen jelent többletköltséget. A kompakt, bájtos megvalósítás csak a 256 bájtos S-boxot és inverzét tárolja, de a $GF(2^8)$ feletti aritmetikát futás közben számolja; a T-táblás megvalósítás gyors, de kb. 10 KB táblát igényel. A dolgozatban a tervezők T-táblás referenciakódját mértük, a bájtos megoldásra pedig összevetést adunk (5.7. alfejezet). A vizsgált STM32L432KC-n nincs hardveres AES-gyorsító, így az algoritmus tisztán szoftveresen fut.

## 3.2. SPECK 128/128

A SPECK algoritmuscsaládot az amerikai Nemzetbiztonsági Ügynökség (NSA) tette közzé 2013-ban, a SIMON családdal együtt [12]. A SIMON elsősorban hardveres, a SPECK szoftveres megvalósításra készült. A családban több blokk- és kulcsméret is szerepel; mi

a 128 bites blokkot és 128 bites kulcsot használó SPECK 128/128 változatot vizsgáljuk.

### 3.2.1. Körfüggvény

A SPECK 128/128 állapota két 64 bites szó ($x$, $y$). A titkosítás 32 körből áll, minden kör a következő ARX-lépést hajtja végre:

$$x \leftarrow ((x \ggg 8) \boxplus y) \oplus K_i, \tag{3.1}$$

$$y \leftarrow (y \lll 3) \oplus x, \tag{3.2}$$

ahol $\ggg$ és $\lll$ a jobbra, illetve balra forgatás, $\boxplus$ az összeadás $\bmod 2^{64}$, $K_i$ pedig az $i$-edik körkulcs. Egy kör tehát két forgatás, egy összeadás és két XOR; a nemlinearitást egyedül a moduláris összeadás adja [12]. Visszafejtéskor ugyanezek a lépések fordított sorrendben, kivonással és ellentétes irányú forgatással futnak.

### 3.2.2. Kulcsgenerálás

A kulcsgenerálás ugyanazt a körfüggvényt használja, mint a titkosítás, csak a kör sorszáma tölti be a kulcs szerepét. A 128 bites kulcs két 64 bites szóra bomlik: $K = (l_0, k_0)$, ahol $k_0$ az első körkulcs. A további körkulcsok:

$$l_{i+1} = (k_i \boxplus (l_i \ggg 8)) \oplus i, \tag{3.3}$$

$$k_{i+1} = (k_i \lll 3) \oplus l_{i+1}. \tag{3.4}$$

Ennek két előnye van: a kulcsgeneráláshoz nem kell külön kód, és a körkulcsok menet közben (on the fly) is előállíthatók, nem kell mindet előre a RAM-ban tárolni.

### 3.2.3. Szoftveres megvalósítás

A SPECK nem használ táblát, végrehajtási ideje nem függ az adattól, és egy 64 bites processzoron minden művelete egyetlen utasítás. Egy 32 bites Cortex-M magon viszont a 64 bites szavakat regiszterpárokban kell kezelni: az összeadás két utasítás (ADDS és ADC), a 64 bites forgatás pedig több eltolásból és kombinálásból áll. Dinu és munkatársai összehasonlító mérésében a SPECK így is a leggyorsabb szoftveres könnyűsúlyú blokktitkosítók közé tartozott [17].

### 3.2.4. Bizalmi kérdések

A SPECK körüli viták elsősorban nem kriptoanalitikai, hanem bizalmi jellegűek. Az eredeti publikáció nem indokolta részletesen a tervezési döntéseket, például a forgatási paraméterek és a körszám megválasztását; ezt a tervezők 2017-ben részben pótolták [13]. A SIMON és a SPECK felvételét a könnyűsúlyú blokktitkosítókat tartalmazó ISO/IEC 29192-2 szabványba 2015-ben javasolták, de 2018 áprilisában elutasították, amiben a tervezővel szembeni bizalmatlanság is szerepet játszott [42]. Szűkebb területen, az RFID-eszközök légi interfészének kriptográfiai csomagjaként viszont mindkettő ISO/IEC-szabvány lett: az ISO/IEC 29167-21 (SIMON) és a 29167-22 (SPECK) első kiadása 2018-ban jelent meg, a jelenleg érvényes második kiadásuk 2026-ban [47, 48]. A nyilvános kriptoanalízis ugyanakkor több mint egy évtized alatt sem talált a teljes körszámú változat ellen gyakorlatban kivitelezhető támadást (6.3. alfejezet).

## 3.3. LEA-128

A *Lightweight Encryption Algorithm* (LEA) a dél-koreai Nemzeti Biztonsági Kutatóintézet (NSRI) fejlesztése, amelyet Hong és munkatársai 2013-ban mutattak be [15]. Elsősorban általános célú, 32 bites processzorokon való gyors szoftveres futásra tervezték. 2019 óta az ISO/IEC 29192-2 szabvány része [16], Dél-Koreában pedig a KCMVP (Korean Cryptographic Module Validation Program) által jóváhagyott algoritmus.

### 3.3.1. Körfüggvény

A LEA a 128 bites blokkot négy 32 bites szóra bontja, $(X_0, X_1, X_2, X_3)$. Egy körben három szó új értéke ARX-műveletekkel áll elő, a negyedik szó pedig az előző $X_0$ lesz; a diffúziót a forgatások és ez a szócsere adják. Egy kör [15]:

$$X_0' = \mathrm{ROL}\big((X_0 \oplus RK_0) \boxplus (X_1 \oplus RK_1),\ 9\big), \tag{3.5}$$

$$X_1' = \mathrm{ROR}\big((X_1 \oplus RK_2) \boxplus (X_2 \oplus RK_3),\ 5\big), \tag{3.6}$$

$$X_2' = \mathrm{ROR}\big((X_2 \oplus RK_4) \boxplus (X_3 \oplus RK_5),\ 3\big), \tag{3.7}$$

$$X_3' = X_0, \tag{3.8}$$

ahol $RK_0, \ldots, RK_5$ a kör hat 32 bites körkulcsszava, $\boxplus$ az összeadás $\bmod 2^{32}$, ROL és ROR pedig a balra, illetve jobbra forgatás. A LEA-128 24 körből áll, így összesen $24 \times 6 = 144$ körkulcsszót használ.

### 3.3.2. A 32 bites szóméret

A LEA és a SPECK legfontosabb gyakorlati különbsége a szóméret. A LEA minden művelete 32 bites, így egy 32 bites magon mindegyik egyetlen utasítás, a forgatás pedig a Cortex-M4 eltolóegysége (barrel shifter) miatt akár egy másik utasításba is beolvasztható. A SPECK 128/128 64 bites szavait ugyanezen a magon csak több utasítással lehet kezelni. A tervezők ezért a LEA-t 32 bites platformokon a SPECK-nél gyorsabbnak várták [15]; hogy ez a mi méréseinkben mennyiben igazolódott, azt az 5. fejezet mutatja meg.

### 3.3.3. Kulcsgenerálás

A LEA-128 kulcsgenerálása négy 32 bites konstansra $(\delta_0, \ldots, \delta_3)$ épül, amelyek a $\sqrt{766965}$ hexadecimális kifejtéséből származnak (766965 a „LEA” betűinek ASCII-kódja). A fő kulcs négy szava $K_0, \ldots, K_3$; az $i$-edik körben

$$T_j = \mathrm{ROL}\big(T_j \boxplus \mathrm{ROL}(\delta_{i \bmod 4},\ i+j),\ r_j\big), \quad j = 0,1,2,3, \tag{3.9}$$

ahol kezdetben $T_j = K_j$, a forgatási paraméterek pedig $r_0 = 1$, $r_1 = 3$, $r_2 = 6$, $r_3 = 11$. A kör hat körkulcsszava $RK = (T_0, T_1, T_2, T_1, T_3, T_1)$. A kulcsgenerálás tehát szintén ARX-műveletekből áll, és menet közben is végrehajtható [15].

## 3.4. Ascon-AEAD128

Az Ascont a Grazi Műszaki Egyetem kutatói, Christoph Dobraunig, Maria Eichlseder, Florian Mendel és Martin Schläffer tervezték. 2014-ben a CAESAR hitelesített titkosítási versenyre nyújtották be, ahol 2019-ben a könnyűsúlyú kategória első számú választása lett, 2023-ban a NIST is kiválasztotta, 2025-ben pedig a NIST SP 800-232 szabvány része lett [9–11]. A többi vizsgált algoritmussal ellentétben nem blokktitkosító, hanem egy permutációra épülő hitelesített titkosítási (AEAD) séma; ugyanerre a permutációra épülnek a család hash- és XOF-függvényei is. A dolgozatban a szabvány AEAD-sémáját, az Ascon-AEAD128-at vizsgáljuk.

### 3.4.1. Szivacs- és duplexkonstrukció

A szivacskonstrukció (sponge construction) egy rögzített permutációra épül, és a nagy belső állapotot két részre osztja: a *rate* ($r$ bit) részen keresztül olvassa be az adatot és adja ki az eredményt, a *capacity* ($c$ bit) rész kívülről közvetlenül soha nem érhető el. A konstrukció biztonsága a capacity méretén múlik. Az Ascon ennek kulcsolt, kétirányú változatát, a *duplexkonstrukciót* használja, amelyben minden adatblokk után lefut a per-

mutáció, és a kimenet is a rate részből származik [9]. Az Ascon belső állapota 320 bites, öt 64 bites szóból áll ($x_0, \ldots, x_4$). Az Ascon-AEAD128-ban a rate 128, a capacity 192 bit.

### 3.4.2. Az Ascon-permutáció

Az Ascon alapeleme egy $p$ permutáció, amelyet az inicializáláskor és a véglegesítéskor $a = 12$, az adatfeldolgozás során az Ascon-AEAD128-ban $b = 8$ körrel alkalmaznak. Egy kör három rétegből áll:

1. **Körkonstans hozzáadása**: egy körönként eltérő konstanst XOR-oz az $x_2$ szóhoz.
2. **Helyettesítő réteg**: 64 párhuzamos, 5 bites S-box. Az S-boxot nem táblázatként, hanem bitenkénti logikai műveletekkel (AND, XOR, NOT) számolják ki egyszerre mind az öt szón (*bitsliced* megvalósítás), így nincs memóriából olvasott tábla, és a végrehajtási idő nem függ az adattól.
3. **Lineáris diffúziós réteg**: minden szót önmaga két elforgatott változatával XOR-oz, például $x_0 \leftarrow x_0 \oplus (x_0 \ggg 19) \oplus (x_0 \ggg 28)$. A forgatási értékek szavanként eltérnek.

### 3.4.3. Az AEAD működése

Az Ascon-AEAD128 128 bites kulcsot, 128 bites nonce-ot és 128 bites hitelesítő címkét használ. Egy hitelesített titkosítás négy fázisból áll [9, 11]:

1. **Inicializálás**: az állapot a 64 bites kezdőérték (IV), a kulcs ($K$) és a nonce ($N$) összefűzése. Ezt 12 körös permutáció követi, majd a kulcsot XOR-ral az állapot végéhez adjuk.
2. **Kapcsolódó adat feldolgozása**: az AD-t 128 bites blokkokra bontjuk, minden blokkot a rate részhez adjuk, és 8 körös permutációt alkalmazunk. Üres AD esetén ez a lépés elmarad. Végül egyetlen bitet (domain separation) átfordítunk az állapotban, ami elválasztja a kapcsolódó adatot a nyílt szövegtől.
3. **Titkosítás**: a nyílt szöveg minden 128 bites blokkját XOR-ral a rate részhez adjuk; az eredmény a titkosított blokk, és egyben a rate új értéke. Az utolsó, kiegészített blokk kivételével minden blokk után 8 körös permutáció következik.
4. **Véglegesítés**: a kulcsot ismét az állapothoz adjuk, 12 körös permutációt alkalmazunk, végül az állapot utolsó 128 bitjét a kulccsal XOR-ozva kapjuk a hitelesítő címkét ($T$).

Visszafejtéskor a fogadó ugyanezt a folyamatot futtatja, kiszámolja a címkét, és összeveti a kapottal. Eltérés esetén a nyílt szöveget nem adja ki, így módosított vagy sérült üzenet nem kerül feldolgozásra.

Fontos következmény, hogy az üzenet hosszától függetlenül minden titkosítás tartalmaz két 12 körös permutációt. Ez üzenetenként fix költség, amely rövid üzeneteknél meghatározóvá válik (5.3. alfejezet).

### 3.4.4. A vizsgált változat

Az Ascon v1.2 specifikáció két AEAD-változatot írt le [9]. Az Ascon-128 rate-je 64 bit, capacity-je 256 bit, és adatfeldolgozáskor $b = 6$ kört használ, az Ascon-128a rate-je 128 bit, capacity-je 192 bit, $b = 8$ körrel. A 128a bájtonként kevesebb permutációs kört futtat (16 bájtonként 8, szemben a 8 bájtonkénti 6 körrel), ezért hosszabb üzeneteken gyorsabb. A NIST SP 800-232 Ascon-AEAD128 sémája az Ascon-128a paramétereit vette át, de kisebb módosításokkal (például más kezdőértékkel és little-endian bájtsorrenddel), így egyik v1.2-es változattal sem bitre azonos [11]. A szabvány a címke rövidítését is megengedi; mi a teljes, 128 bites címkét használtuk.

A dolgozatban a szabványos Ascon-AEAD128-at mértük, mert új fejlesztésekben ezt kell használni; a tervezők referenciakódja és hivatalos tesztvektorai (KAT) is elérhetők hozzá.

## 3.5. Az algoritmusok összevetése

A 3.1. táblázat a négy algoritmus fő szerkezeti jellemzőit foglalja össze.

3.1. táblázat. A vizsgált algoritmusok szerkezeti jellemzői

| Jellemző | AES-128 | SPECK 128/128 | LEA-128 | Ascon-AEAD128 |
|---|---|---|---|---|
| Típus | blokktitkosító | blokktitkosító | blokktitkosító | AEAD |
| Szerkezet | SPN | ARX, 2 szó | ARX, 4 szó | duplex (szivacs) |
| Blokkméret / rate | 128 bit | 128 bit | 128 bit | 128 bit (rate) |
| Kulcsméret | 128 bit | 128 bit | 128 bit | 128 bit |
| Körök száma | 10 | 32 | 24 | 12 / 8 |
| Szóméret | 8 bit (T-táblával 32) | 64 bit | 32 bit | 64 bit |
| S-box | 8 bites, 256 elem | – | – | 5 bites, bitsliced |
| Nemlinearitás | S-box | mod. összeadás | mod. összeadás | S-box (logikai) |
| Szabvány | FIPS 197 | ISO/IEC 29167-22[†] | ISO/IEC 29192-2 | NIST SP 800-232 |

[†]Csak RFID-légi interfészhez; az általános ISO/IEC 29192-2-be nem került be.

Az AES a táblás S-boxra épülő SPN-t képviseli, amely 32 bites processzoron T-táblákkal valósítható meg hatékonyan. A SPECK és a LEA az S-box nélküli, aritmetikai nemlinearitású ARX-irányzat két változata, amelyek szóméretben és ágszámban térnek

el. Az Ascon-AEAD128 a titkosítást és a hitelesítést egyetlen, permutációra épülő konstrukcióban valósítja meg. Mind a négy 128 bites kulcsot használ, ami azonos alapot ad a biztonsági szintek összevetéséhez. A három blokktitkosító 128 bites blokkmérete mellett a születésnapi korlát (6.2.5. alfejezet) $2^{64}$ blokk, ami IoT-alkalmazásokban nem jelent valódi korlátot.

# 4. fejezet

# Implementáció és mérési módszer

Ez a fejezet a mérési platformot, az algoritmusok megvalósítását és a közös mérőprogramot írja le, olyan részletességgel, hogy a mérések megismételhetők legyenek.

## 4.1. A mérési platform

A méréseket egy NUCLEO-L432KC fejlesztőkártyán végeztük. A kártyán lévő STM32L432KC mikrovezérlő ARM Cortex-M4 magos, legfeljebb 80 MHz-es órajellel, 256 KB flash- és 64 KB SRAM-memóriával. A Cortex-M4 32 bites mag: az összeadás, az XOR és a 32 bites forgatás egy-egy utasítás, 64 bites aritmetikája viszont nincs. A kártya beépített ST-LINK programozója a programletöltésre és a soros kommunikációra is szolgált.

A rendszerórajel 80 MHz, amelyet a belső MSI-oszcillátor (4 MHz) jeléből a PLL állít elő. Ezen a frekvencián a flash olvasása 4 várakozási ciklust (wait state) igényel. Ennek hatását az STM32L4 család ART-gyorsítója mérsékli, amely egy 1 KB-os utasításgyorsítótárból és egy 256 bájtos adat-gyorsítótárból áll, és mindkettő a flash-memória elé kapcsolódik [38]. A Cortex-M4 magnak saját gyorsítótára nincs, így az ART az egyetlen gyorsítótár a rendszerben. Mivel ezek a beállítások közvetlenül befolyásolják a mért ciklusszámokat, a 4.1. táblázat rögzíti őket.

4.1. táblázat. A mérési környezet

| **Elem** | **Érték** |
|---|---|
| Fejlesztőkártya | NUCLEO-L432KC |
| Mikrovezérlő | STM32L432KC (ARM Cortex-M4) |
| Órajel | 80 MHz (MSI 4 MHz → PLL), 1-es feszültségtartomány |
| Flash várakozási ciklusok | 4 (`FLASH_LATENCY_4`) |
| ART-gyorsító | utasítás- és adat-gyorsítótár be, előolvasás (prefetch) ki |
| Fejlesztőkörnyezet | STM32CubeIDE 2.1.1 |
| Fordító | arm-none-eabi-gcc 14.3.1 (GNU Tools for STM32) |
| Fordítási kapcsolók | `-mcpu=cortex-m4 -mthumb -mfloat-abi=hard -ffunction-sections -fdata-sections`, C11 |

## 4.2. A forráskód szerkezete

A mérőprogram egyetlen STM32CubeIDE-projekt. A hardver inicializálását (órajel, GPIO, USART2) a CubeMX által generált kód végzi. Az önellenőrzés, a mérési ciklus, a statisztika és a kiírás a `main.c`-ben van, az algoritmusok egy-egy fájlpárban (`rijndael.c/h`, `speck.c/h`, `lea.c/h`, `ascon_aead128.c/h`, az összevetéshez `aes.c/h`), a fájl elején a forrás pontos megnevezésével. Egyik algoritmus sem használ dinamikus memóriafoglalást.

## 4.3. Az algoritmusok megvalósítása

Mind a négy algoritmus hordozható C-kód, assembly és platformspecifikus gyorsítás nélkül. Az összehasonlíthatóság érdekében mindegyiknél a tervezők, illetve a szabványt gondozó szervezet nyilvános referenciakódját használtuk, saját algoritmuskódot nem írtunk. A 4.2. táblázat mutatja, melyik kód honnan származik, és hogyan ellenőriztük a helyességét. Hasonló összehasonlító méréseket korábban többek között Dinu és munkatársai [17], valamint Cazorla és munkatársai [18] végeztek; a mi vizsgálatunk ezekhez képest a szabványos Ascon-AEAD128-at és a fordítói optimalizáció hatását is magában foglalja.

4.2. táblázat. Az implementációk forrása és ellenőrzése

| **Algoritmus** | **Alap** | **Helyességellenőrzés** |
|---|---|---|
| AES-128 | a tervezők optimalizált C-kódja, `rijndael-alg-fst.c` v3.0 [33] (public domain), az algoritmusrész változatlan | FIPS 197, C.1 [4] és NIST SP 800-38A, F.1.1 ECB-vektor [32] |
| SPECK 128/128 | NSA *Implementation Guide* v1.1, 6. fejezet [14], kisebb módosításokkal | az útmutató tesztvektora |
| LEA-128 | a KISA hivatalos C-forráskódja, v1.3 [31], az algoritmusrész változatlan | a szabvány tesztvektora [30] |
| Ascon-AEAD128 | a tervezők `ascon-c` könyvtára [36], `asconaead128/bi32` (CC0), az algoritmusrész változatlan | a hivatalos KAT-fájl mind az 1089 vektora (PC-n), ebből kettő a kártyán is |
| AES-128 (összevetés) | `tiny-AES-c` [35] (The Unlicense), AES-128-ra szűkítve | SP 800-38A, F.1.1 |

### 4.3.1. AES-128

Az AES-hez Rijmen, Bosselaers és Barreto optimalizált ANSI C kódját, a `rijndael-alg-fst.c` 3.0-s változatát használtuk [33]. Ez a tervezők saját referenciakódja, amelyre

számos elterjedt könyvtár épül, például az OpenSSL hordozható C-nyelvű AES-e és a FreeBSD kernelének AES-kódja. Az algoritmusrészt változtatás nélkül vettük át a FreeBSD forrásfájljából; csak a típusdefiníciókat igazítottuk az `stdint.h`-hoz, és a kernelspecifikus paraméterellenőrzést cseréltük üres makróra.

A kód a 3.1.1. alfejezetben leírt T-táblás megoldást követi, a tíz 1 KB-os tábla a flash-memóriában van. A titkosító és a visszafejtő körkulcsokat két külön függvény állítja elő (`rijndaelKeySetupEnc`, `rijndaelKeySetupDec`). A körök teljes kibontását bekapcsoló `FULL_UNROLL` makrót nem definiáltuk, így a kód alapbeállítása szerint egy ciklus fut, iterációnként két körrel. Összevetésként a beágyazott projektekben elterjedt `tiny-AES-c` könyvtárat [35] is mértük (5.7. alfejezet); a dolgozat minden más AES-adata a referenciakódra vonatkozik.

### 4.3.2. SPECK 128/128

A SPECK-hez az NSA nyilvános megvalósítási útmutatójának referenciakódját használtuk [14]. A körfüggvényt két makró valósítja meg (`ER64` titkosításhoz, `DR64` visszafejtéshez), a kulcsgenerálás ugyanazt az `ER64`-et hívja a kör sorszámával. A makrók és a függvénytörzsek lényegében változatlanok, három apró eltéréssel: a csak olvasott paraméterek `const` minősítést kaptak, a visszafejtő függvény a titkosítóhoz hasonlóan előbb a bemenetet, utána a kimenetet kapja, a bájtátalakítást pedig `memcpy` végzi. Ez utóbbi a mérésekben nem is fut, mert a mérőciklus a 8 bájtra igazított puffereket közvetlenül 64 bites szavakként adja át. A 32 körkulcsot (256 bájt) előre kiszámoljuk.

### 4.3.3. LEA-128

A LEA-hoz a dél-koreai KISA (Korea Internet & Security Agency) hivatalos C-forráskódját, a v1.3-as *LEA C Standalone* csomagot használtuk [31]. A csomagban üzemmódok (ECB, CBC, CTR, GCM, CCM, CMAC) és x86-os SIMD-változatok is vannak, a méréshez csak a blokkművelet és a kulcsgenerálás kellett. A `lea_core.c` teljes egészében, a `lea_base.c`-ből a `lea_set_key`, a `lea_locl.h`-ból a forgató és a bájtsorrend-kezelő makrók kerültek a `lea.c`-be, a `config.h` és a deklarációk a `lea.h`-ba, változtatás nélkül; csak a helyi `#include` sorok maradtak el.

A KISA kódja a 24 kört ciklus nélkül, teljesen kibontva tartalmazza, a 192 és 256 bites kulcshoz tartozó további 4, illetve 8 kört pedig feltételes ágakban. A forgatások makrók, így függvényhívás egyik szinten sincs. A kulcsgenerálás is teljesen kibontott, és körönként mind a hat körkulcsszót külön eltárolja (a $RK_1 = RK_3 = RK_5$ egyezést nem használja ki), 128 bites kulcsnál összesen 144 szót. A blokkot `unsigned int` mutatón keresztül olvassa és írja, ezért a puffereknek 4 bájtos határra igazítottnak kell lenniük: optimalizált kódban a fordító dupla szavas betöltést (`LDRD`) használ, amely a Cortex-M4-

en igazítatlan címen hibát okoz. A mérőprogram pufferei 8 bájtos határra igazítottak.

### 4.3.4. Ascon-AEAD128

Az Ascon-AEAD128-hoz a tervezők hivatalos `ascon-c` könyvtárát használtuk [36], az `asconaead128` séma `bi32` változatát. A könyvtárban ugyanahhoz a sémához több megvalósítás is van, a `ref` referenciakódtól a Cortex-M-re írt assemblyig. Ezek közül a `bi32` 32 bites processzorokra készült, tiszta C-kód, így a többi algoritmushoz hasonlóan hordozható C-kódot mérünk vele.

A `bi32` a 64 bites állapotszavakat bit-összefésült (bit-interleaved) alakban tárolja: minden szót két 32 bites félre bont, az egyikbe a páros, a másikba a páratlan helyiértékű bitek kerülnek. Így a permutáció 64 bites forgatásai két 32 bites forgatásra bomlanak, amelyeket a Cortex-M4 egy-egy utasítással, gyakran egy másik utasításba olvasztva végez el. A könyvtár 14 forrásfájlját függőségi sorrendben, változtatás nélkül egymás után másoltuk az `ascon_aead128.c`-be, csak a helyi `#include` sorok maradtak el; az `api.h` és a `crypto_aead.h` tartalma az `ascon_aead128.h`-ba került. A konfiguráció a könyvtár alapbeállítása: az üzemmód lépései beágyazódnak a hívó függvénybe, a 12 és a 8 körös permutáció pedig két külön, kibontott körökkel megírt függvény (`P12`, `P8`).

A mérőprogram a NIST könnyűsúlyú pályázatán előírt `crypto_aead_encrypt` és `crypto_aead_decrypt` függvényt hívja [37]; a címke ellenőrzése konstans idejű. A helyességet PC-n a hivatalos KAT-fájl mind az 1089 vektorával ellenőriztük; ezek közül kettőt (kapcsolódó adat nélkül és 16 bájt kapcsolódó adattal) a mikrovezérlő is lefuttat induláskor.

## 4.4. A mérési keretrendszer

### 4.4.1. Önellenőrzés és bemenetek

A program induláskor minden algoritmusra lefuttatja a 4.2. táblázatban felsorolt tesztvektorokat, majd visszafejti az eredményt, és összeveti az eredeti nyílt szöveggel. Az Ascon-AEAD128-nál azt is ellenőrzi, hogy egy bitjében módosított titkosított szöveget a visszafejtés elutasít. Ha bármelyik ellenőrzés hibát jelez, a mérés el sem indul.

A mérésekben minden algoritmus ugyanazt a 128 bites kulcsot kapja (hexadecimálisan `0123456789ABCDEF`, kétszer egymás után), a nyílt szöveg $i$-edik bájtja $i$ mod 256. Az üzenetméretek 16, 64, 256 és 1024 bájt. Az Ascon-AEAD128 128 bites nonce-a csupa nulla, a kapcsolódó adat üres.

### 4.4.2. Ciklusmérés a DWT-egységgel

A Cortex-M4 hibakereső egysége, a DWT (Data Watchpoint and Trace) egy 32 bites ciklusszámlálót (`CYCCNT`) tartalmaz, amely minden órajelciklusban eggyel nő [39]. A `DWT_Init()` engedélyezi a nyomkövetést és elindítja a számlálót, egy művelet mérését pedig a `MEASURE` makró végzi (4.1. kódrészlet).

```
void DWT_Init(void) {
    DEMCR |= DEMCR_TRCENA;          /* nyomkovetes engedelyezese */
    DWT_CYCCNT = 0;
    DWT_CTRL |= DWT_CR_CYCCNTENA;   /* szamlalo inditasa */
}

#define MEASURE(ALGO, OP, SIZE, CALL)                          \
    do {                                                       \
        CALL;                                /* bemelegites */ \
        __disable_irq();                                       \
        uint32_t t0_ = DWT_CYCCNT;                             \
        for (int k_ = 0; k_ < ITERATIONS; k_++) { CALL; }      \
        uint32_t t1_ = DWT_CYCCNT;                             \
        __enable_irq();                                        \
        Record_Result((ALGO), (OP), (SIZE), (t1_ -
t0_) / ITERATIONS); \
    } while (0)
```

4.1. Kódrészlet. A ciklusszámláló indítása és egy művelet mérése

Egy mérés menete: egy nem mért, bemelegítő hívás (ez tölti fel az ART gyorsítótárait), majd tiltott megszakítások mellett a művelet 100-szori ismétlése, végül az egy hívásra jutó átlag. A megszakítások tiltása kizárja, hogy a HAL 1 ms-os SysTick-megszakítása belekerüljön a mért időbe. A mért érték a mérőciklus és a függvényhívás költségét is tartalmazza. Ezt a program induláskor külön megméri: optimalizált szinteken az üres ciklus iterációnként kb. 3,1, egy üres függvényhívással együtt kb. 7,2 ciklus, az Ascon-AEAD128 hívásával azonos, 9 paraméteres üres hívással 37–40 ciklus (`-O0` mellett 9,2, 22,2 és 65,3). Ez a legrövidebb mért műveleteknél is legfeljebb kb. 1,5 %, ezért az eredményekből nem vontuk le.

A 32 bites számláló 80 MHz-en kb. 53,7 másodpercenként fordul át. Az előjel nélküli kivonás egy átfordulást helyesen kezel; a leghosszabb mért szakasz (a `tiny-AES-c` optimalizálás nélküli visszafejtése 100-szor 1024 bájton) kb. 22 másodperc, jóval a korlát alatt.

### 4.4.3. Ismétlések, statisztika és kiírás

A teljes mérési kört (kulcsgenerálás, majd titkosítás és visszafejtés mind a négy méreten) a program tízszer egymás után lefuttatja. Minden mért értékre a tíz kör mediánját, minimumát és maximumát, valamint a $(\max - \min)/\text{medián}$ relatív eltérést számolja. Az 5. fejezetben minden szám a medián.

A mikrovezérlő az eredményeket az USART2 perifériáján (115 200 baud, az ST-LINK virtuális soros portja) küldi a számítógépre. A kiírás mindig a mért szakaszokon kívül történik: a program körönként CSV-sorokat (futás sorszáma, algoritmus, művelet, üzenetméret, ciklusszám), a végén pedig összesítő táblázatot küld. A naplókat a `graphics.py` Python-szkript (matplotlib) dolgozza fel, ez készíti a táblázatok adatait és az ábrákat is.

## 4.5. Az üzemmódok megválasztása

A három blokktitkosítót ECB módban mértük. Az ECB a blokkműveleten felül semmilyen láncolást vagy számlálókezelést nem végez, így a mért költség csak a blokktitkosító sajátja. Az ECB-ciklus mindhárom algoritmusnál a `main.c`-ben van, azonos szerkezetben: minden 16 bájtos blokkra egyszer hívja a referenciakód blokkfüggvényét. Biztonsági okból (2.1.2. alfejezet) az ECB éles rendszerben nem használható, itt kizárólag mérési célt szolgál. A `tiny-AES-c`-nél a CTR-módot is mértük (5.7. alfejezet).

Az Ascon-AEAD128-at natív AEAD-módjában mértük, üres kapcsolódó adattal. Költsége így a hitelesítő címke kiszámítását (visszafejtéskor ellenőrzését) is tartalmazza, amit a blokktitkosítókkal való összevetéskor figyelembe kell venni.

## 4.6. Fordítási beállítások

A teljes programot hat optimalizációs szinttel fordítottuk és mértük le [40]: `-O0` (optimalizálás nélkül, a fejlesztés alapbeállítása), `-O2` (általános sebességoptimalizálás a méret és a sebesség közötti kompromisszumot igénylő lépések nélkül), `-O3` (erősebb, többek között agresszívebb beágyazással és ciklusbontással), `-Ofast` (`-O3` a lebegőpontos szabványmegfelelés lazításával), `-Os` (kódméret-optimalizálás) és `-Oz` (erős kódméret-optimalizálás). A szintet mindig a projekt egészére állítottuk, hogy minden forrásfájl azonos beállítással forduljon. A program induláskor kiírja a GCC előre definiált makróiból megállapítható szintcsoportot (`-O0`, `-O1`/`-O2`/`-O3`, `-Ofast`, `-Os`/`-Oz`) és az ART-gyorsító beállítását. A makrók az `-O2` és az `-O3` között nem tesznek különbséget, ezért a pontos szintet a projektbeállítás és a naplófájl neve rögzíti. Szintváltás után a projektet mindig teljesen újrafordítottuk (Clean, majd Build).

Az `-Os` és az `-Oz` mellett a mért ciklusszámok minden mérési pontban megegyeztek, és a lefordított objektumfájlok összevetése szerint a vizsgált kódra Cortex-M4-en bájtra

azonos gépi kód keletkezett. A továbbiakban ezért a kettőt együtt, `-Os/-Oz` jelöléssel szerepeltetjük.

# 5. fejezet

# Teljesítménymérések és eredmények

Ez a fejezet a 4. fejezetben leírt mérőprogrammal kapott eredményeket mutatja be. Minden érték tíz mérési kör mediánja. A mérések gyakorlatilag determinisztikusak: a körök közötti legnagyobb relatív eltérés az összes mérési pontot tekintve 0,001 % volt, a legtöbb pontban nulla. Minden kör ugyanazt a bemenetet dolgozza fel, megszakítás nem zavarja, és a bemelegítés után a gyorsítótár állapota is azonos. A SPECK értékei ciklusra egyeztek a mérőprogram korábbi, más szerkezetű változatával végzett mérésekkel. A kis szórás a mérés ismételhetőségét igazolja, abszolút pontosságát önmagában nem. Ez kontrollált benchmarkkörnyezet: valós alkalmazásban a megszakítások és a hideg gyorsítótár a mértnél nagyobb költséget is okozhatnak. Az ábrákon a SPECK-128 a SPECK 128/128-at, az AES-128 a tervezők referenciakódját jelöli. Ahol egy ábra a `tiny-AES-c`-t is mutatja, az az 5.7. alfejezet összevetéséhez tartozik.

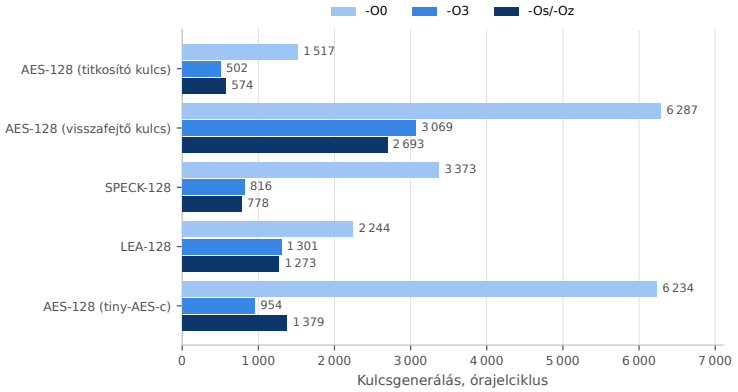
## 5.1. A kulcsgenerálás költsége

Az 5.1. táblázat a kulcsgenerálás ciklusszámát mutatja minden szinten, az 5.1. ábra pedig három jellemző szintet. Az AES-nél a titkosító és a visszafejtő körkulcsok előállítását külön mértük, mert a referenciakódban ez két külön függvény (4.3. alfejezet).

5.1. táblázat. A kulcsgenerálás ciklusszáma optimalizációs szintenként

| **Algoritmus** | `-O0` | `-O2` | `-O3` | `-Ofast` | `-Os/-Oz` |
|---|---|---|---|---|---|
| AES-128, titkosító kulcs | 1 517 | 625 | 502 | 501 | 574 |
| AES-128, visszafejtő kulcs | 6 287 | 2 756 | 3 069 | 3 076 | 2 693 |
| SPECK-128 | 3 373 | 816 | 816 | 816 | 778 |
| LEA-128 | 2 244 | 1 301 | 1 301 | 1 301 | 1 273 |

Az `-O0`-ról `-O3`-ra váltva a kulcsgenerálás az AES titkosító kulcsánál 3,0-szor, a visszafejtőnél 2,0-szer, a SPECK-nél 4,1-szer, a LEA-nál csak 1,7-szer gyorsult. A KISA LEA-kódjában a kulcsgenerálás teljesen kibontott, a forgatások makrók, így optimalizálás nélkül sincs benne függvényhívás, és a fordítónak is kevés javítanivaló marad. Sebességre




5.1. ábra. A kulcsgenerálás ciklusszáma `-O0`, `-O3` és `-Os`/`-Oz` mellett. A `tiny-AES-c` egyetlen körkulcskészletet használ mindkét irányhoz. Az Ascon-AEAD128 nem szerepel, mert nincs külön kulcsgenerálási lépése.

optimalizált kódban a titkosító AES-kulcs a legolcsóbb (501–625 ciklus), a SPECK 816, a LEA 1 301 ciklus. `-O3` mellett a fordító az AES kulcsgeneráló ciklusát kibontja (a függvény 500 helyett 1 728 bájt), ezért gyorsabb. A LEA kulcsgenerálása 128 bites kulccsal 442 utasításból álló, 1,3 KB-os egyenes kódrészt futtat végig, amelyben a körkonstansokat is a flash-memóriából olvassa; a SPECK ugyanezt egy 110 bájtos függvényben, ciklussal végzi. Hogy a LEA-nál miért a kód hossza számít, azt a 5.6.4. alfejezet mutatja meg.

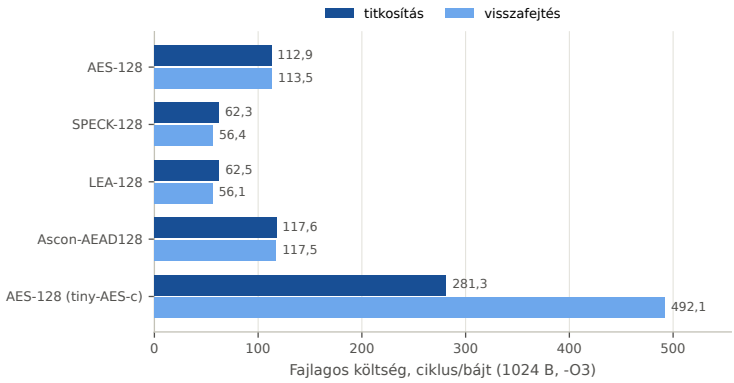
A visszafejtő AES-kulcs 2 693–3 076 ciklusba kerül, a titkosító kulcs 4,1–6,1-szeresébe, mert a titkosító körkulcsok után a kilenc belső körkulcsra az inverz MixColumns-t is alkalmazni kell. Ez az ára annak, hogy maga a visszafejtés ugyanolyan gyors, mint a titkosítás (5.2. alfejezet).

Az Ascon-AEAD128 nem használ külön kulcskiterjesztést: a kulcsot az inicializáláskor és a véglegesítéskor keveri be (3.4.3. alfejezet), így ez a költség a titkosítás mért idejében jelenik meg. Gyakorlati szempontból a kulcsgenerálás akkor számít, ha a kulcs gyakran változik. `-O3` mellett a SPECK kulcsgenerálása egy blokk titkosításának kb. 0,8-szorosa, a LEA-é 1,3, az AES titkosító kulcsáé 0,3, a visszafejtőé 1,7 blokk.

## 5.2. Titkosítás és visszafejtés

A 5.2. ábra a fajlagos költséget (ciklus/bájt, cycles per byte, CPB) mutatja 1024 bájtos üzeneten `-O3` mellett. A többi szint értékei az 5.6. ábrán láthatók.

`-O3` mellett a két ARX-algoritmus a leggyorsabb, és fajlagos költségük szinte azonos: titkosításkor 62,3 (SPECK) és 62,5 (LEA), visszafejtéskor 56,4 és 56,1 ciklus/bájt. Az AES titkosításkor 1,8-szer, visszafejtéskor 2,0-szer drágább náluk. Az Ascon-AEAD128 117,6 ciklus/bájttal a leglassabb, de csak 4 %-kal drágább az AES-nél, és ebben a hitelesítés is benne van (5.3. alfejezet).




5.2. ábra. Fajlagos költség 1024 bájtos üzeneten, `-O3` mellett. Az alsó sor a `tiny-AES-c` (5.7. alfejezet).

Az AES két iránya a referenciakódban gyakorlatilag azonos költségű, az eltérés minden szinten 0,6 % alatti. Ez a T-táblás megvalósítás következménye: a visszafejtés ugyanannyi táblaolvasást és XOR-t végez, mint a titkosítás, az inverz MixColumns költsége pedig a visszafejtő kulcs egyszeri előállításába került (5.1. alfejezet). Az Ascon-AEAD128 két iránya szintén azonos költségű (0,4 %-on belül), mert a duplexkonstrukcióban mindkét irány ugyanazt a permutációt futtatja. A SPECK visszafejtése minden szinten 9–10 %-kal gyorsabb a titkosításnál. A generált kódban a visszafejtő ciklus körönként 19, a titkosító 21 utasítás, és ennek aránya (1,105) megegyezik a mért ciklusszámok arányával. A LEA-nál `-O0` mellett a két irány gyakorlatilag azonos, optimalizált kódban a visszafejtés 10–11 %-kal gyorsabb. Ennek oka nem az utasítások száma, ezt a 5.6.4. alfejezet vizsgálja.

## 5.3. Skálázódás az üzenetmérettel

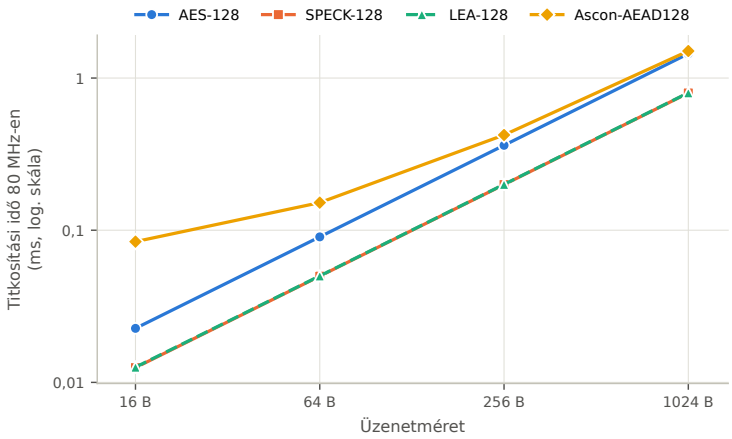
A 5.3. ábra a titkosítás idejét mutatja az üzenetméret függvényében, az 5.2. táblázat pedig a mért ciklusszámokat, mindkettő `-O3` mellett.

5.2. táblázat. A titkosítás ciklusszáma az üzenetméret függvényében (`-O3`)

| **Algoritmus** | **16 B** | **64 B** | **256 B** | **1024 B** |
|---|---|---|---|---|
| AES-128 | 1 812 | 7 233 | 28 907 | 115 619 |
| SPECK-128 | 1 000 | 3 991 | 15 955 | 63 811 |
| LEA-128 | 1 007 | 4 007 | 16 007 | 64 007 |
| Ascon-AEAD128 | 6 731 | 12 146 | 33 806 | 120 446 |

A blokktitkosítók költsége egyenesen arányos az üzenet hosszával: ECB módban minden 16 bájtos blokk ugyanannyiba kerül, így a CPB minden méreten gyakorlatilag állandó (az AES-é 16 bájton 113,3, 1024 bájton 112,9). A logaritmikus ábrán ezért párhuzamos egyeneseket látunk.

Az Ascon-AEAD128 viselkedése eltér ettől: CPB-je 16 bájton 420,7, 1024 bájton




5.3. ábra. A titkosítás ideje az üzenetméret függvényében (`-O3`, 80 MHz, mindkét tengely logaritmikus). A SPECK és a LEA görbéje szinte fedi egymást.

117,6. A négy mérési pont `-O3` mellett ciklusra pontosan illeszkedik a

$$C_{\text{Ascon}}(n) = 4\,926 + 1\,805 \cdot \frac{n}{16} \tag{5.1}$$

egyenesre, ahol $n$ az üzenet hossza bájtban. Egy 16 bájtos blokk tehát 1 805 ciklusba kerül, ebben egy 8 körös permutáció, a blokk bit-összefésülése, az állapothoz adása és a titkosított blokk kiírása van. A tengelymetszet az üzenettől független rész: a két 12 körös permutáció, a kulcs és a nonce betöltése és a címke előállítása. 16 bájtnál $12+8+12 = 32$ permutációs kör fut, 1024 bájtnál 536; mindkét tag kb. 200 ciklusos körköltséggel egyeztethető össze.

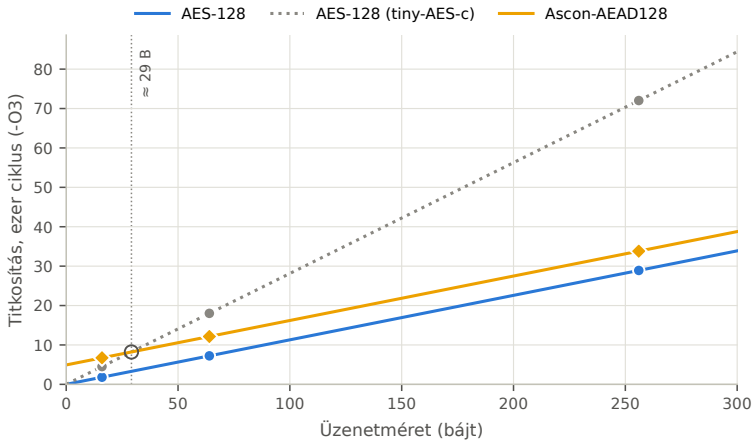
A csak titkosító AES-szel összevetve az Ascon-AEAD128 egyik mért méreten sem gyorsabb, de a különbség a hosszal gyorsan csökken: 16 bájton 3,7-szer, 64 bájton 1,7-szer, 256 bájton 1,17-szer, 1024 bájton már csak 1,04-szer drágább (5.4. ábra). A két egyenes meredeksége szinte azonos (112,8 és 112,9 ciklus/bájt), így hosszú üzeneteken a különbség lényegében az Ascon fix költsége. `-Os`/`-Oz` mellett az Ascon-AEAD128 bájtonkénti költsége 130,0, az AES-é 108,2 ciklus, ott 1024 bájton 1,25-szörös a különbség.

Az összevetés azonban nem azonos szolgáltatások között történik: az Ascon-AEAD128 a hitelesítő címkét is kiszámítja, az ECB módú AES nem. Egy hitelesítést is nyújtó AES-alapú séma ennél drágább. CCM módban például blokkonként két AES-hívás kell (egy a CBC-MAC-hez, egy a CTR-titkosításhoz), és üres kapcsolódó adat mellett is két további: a CBC-MAC kezdőblokkja és a címke titkosítása. A mért blokkköltséggel (`-O3` mellett kb. 1 806 ciklus) ez kb.

$$C_{\text{CCM}}(n) \approx 3\,613 + 225{,}8 \cdot n \tag{5.2}$$

ciklus. Ennek fix költsége kisebb, meredeksége viszont kétszerese az Ascon-AEAD128-énak, így a két egyenes kb. 12 bájtnál metszi egymást. Az Ascon-AEAD128 tehát már

a legrövidebb mért, 16 bájtos üzeneten is olcsóbb (6 731, illetve kb. 7 226 ciklus), 1024 bájton pedig kb. feleannyiba kerül. `-Os`/`-Oz` mellett ($3\,463 + 216{,}4 \cdot n$, illetve $5\,692 + 130{,}0 \cdot n$) a határ kb. 26 bájt. Ez becslés: a CCM-et nem mértük, és a modellből hiányzó XOR-ok, számlálókezelés és formázás a CCM-et tovább drágítanák, vagyis a valós határ inkább alacsonyabb.




5.4. ábra. Az Ascon-AEAD128 és az AES-128 titkosítási költsége rövid üzeneteken (`-O3`); a vonalak a mérési pontokra illesztett egyenesek. A pontozott vonal a `tiny-AES-c`, amelynek egyenese kb. 29 bájtnál metszi az Ascon-AEAD128-ét (5.7. alfejezet).

## 5.4. Feldolgozási sebesség

A feldolgozási sebesség (throughput) azt mutatja, hány bájtot tud a mikrovezérlő másodpercenként titkosítani vagy visszafejteni; ez nem azonos a kommunikációs csatorna átviteli sebességével. Az órajelből és a fajlagos költségből adódik: $v = f_{\mathrm{CLK}}/\mathrm{CPB}$, ahol 1 kB = 1000 B. Az 1024 bájtos üzenetre vonatkozó értékeket az 5.3. táblázat mutatja a gyakorlatban leginkább használt három szinten.

5.3. táblázat. Feldolgozási sebesség 1024 bájtos üzeneten, 80 MHz-en (kB/s; T: titkosítás, V: visszafejtés)

| **Algoritmus** | `-O2` | | `-O3` | | `-Os`/`-Oz` | |
|---|---|---|---|---|---|---|
| | T | V | T | V | T | V |
| AES-128 | 709 | 705 | 709 | 705 | 739 | 740 |
| SPECK-128 | 1 284 | 1 419 | 1 284 | 1 419 | 1 283 | 1 417 |
| LEA-128 | 1 282 | 1 433 | 1 280 | 1 427 | 1 277 | 1 421 |
| Ascon-AEAD128 | 680 | 681 | 680 | 681 | 590 | 592 |

A legnagyobb feldolgozási sebességet a LEA és a SPECK visszafejtése érte el (1 417–1 433 kB/s), titkosításban a kettő 1 277–1 284 kB/s-ot. Az AES és az Ascon-AEAD128 ennek kb. a felét dolgozza fel, az utóbbi a hitelesítéssel együtt.

## 5.5. Kódméret és memóriaigény

A kódméretet és a veremigényt a mért buildekből határoztuk meg (5.4. táblázat). A flash-igény a linker térképfájljából (`.map`) adódik, a nem használt függvényeket eltávolító `-gc-sections` linkelés után, a veremigény a GCC `-fstack-usage` és `-fcallgraph-info` kapcsolójának kimenetéből, a hívási lánc mentén összegezve; a gépi kódot ez a két kapcsoló nem változtatja meg. Az értékeket egy saját Python-szkript (`memoria.py`) gyűjti ki.

5.4. táblázat. Flash-igény (kód és konstansok) és legnagyobb veremhasználat bájtban, a mért függvényekre

| **Algoritmus** | **Flash (B)** | | | | **Verem (B)** | | | |
|---|---|---|---|---|---|---|---|---|
| | `-O0` | `-O2` | `-O3` | `-Os` | `-O0` | `-O2` | `-O3` | `-Os` |
| AES-128* | 14 568 | 12 940 | 14 168 | 12 844 | 72 | 96 | 96 | 96 |
| SPECK-128 | 778 | 296 | 296 | 270 | 88 | 32 | 32 | 20 |
| LEA-128† | 16 794 | 10 652 | 10 652 | 10 640 | 56 | 20 | 20 | 20 |
| Ascon-AEAD128 | 161 270 | 10 816 | 10 816 | 11 520 | 13 312 | 208 | 208 | 248 |

Az `-Oz` értékei az `-Os`-éval, az `-Ofast` értékei az `-O3`-éval azonosak. A veremigény a könyvtári `memcpy` nélkül értendő.
*Ebből 10 280 bájt a tíz T-tábla és a körkonstansok. A titkosító és a visszafejtő kulcsgenerálást, valamint a 192 és 256 bites kulcs kezelését is tartalmazza.
†A 192 és 256 bites kulcs kezelését is tartalmazza.

A kulcsfüggő adatok a RAM-ban külön helyet foglalnak: az AES titkosító és visszafejtő körkulcsai egyenként 176 bájtot, a SPECK 32 körkulcsa 256 bájtot, a LEA kulcsstruktúrája 772 bájtot, amelyből 128 bites kulcsnál 576 bájt van használatban. Az Ascon-AEAD128-nak nincs tárolt körkulcsa, a 40 bájtos állapot és a kulcs a veremben van, a táblázat veremértéke ezeket már tartalmazza. A mérőprogram puffereit nem számoltuk az algoritmusokhoz.

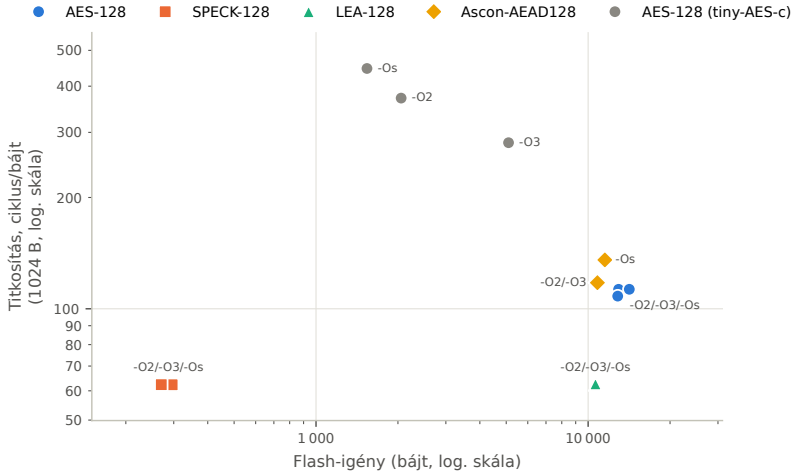
Az AES flash-igénye minden szinten 12,8–14,6 KB, ebből 10,3 KB a táblák, maga a kód 2,6–4,3 KB. Ha csak a titkosító irány kell (például CTR-re épülő módban), a visszafejtő táblák és függvények elhagyhatók, és a flash-igény kb. 6,5–7,8 KB-ra csökken.

A LEA flash-igénye az optimalizált szinteken 10,6–10,7 KB. Ennek kb. 70 %-a (`-O3` mellett 7,4 KB) a teljesen kibontott kulcsgenerálás, amely mindhárom kulcsméretet kezeli; a titkosító és a visszafejtő függvény 1,7, illetve 1,5 KB. Az Ascon-AEAD128 flash-igénye `-O2` és `-O3` mellett 10,8 KB, `-Os` mellett 11,5 KB. A `P12` és a `P8` permutáció kibontott körökkel `-O3` mellett kb. 2,6, illetve 1,8 KB, a titkosító és a visszafejtő függvény, amelybe az üzemmód lépései és a bit-összefésülés beágyazódnak, egyenként kb. 3,1 KB. Méretre optimalizálva a kód nem lesz kisebb, mert a fordító a permutáció állapotának egy részét a veremben tartja, és ehhez több utasítás kell (5.6. alfejezet). `-O0` mellett a kód 161,3 KB, a verem 13,3 KB. A könyvtár a körfüggvényt és a segédfüggvényeket `always_inline` attribútummal jelöli, ezért a GCC optimalizálás nélkül is beágyazza

őket, de minden beágyazott példány változóinak külön veremhelyet foglal, és minden köztes értéket a memóriába ír; egyedül a `P12` kódja így 75,1 KB. A kártya 256 KB-os flash-memóriájába ez még belefér, egy kisebbe már nem.

A SPECK kódja minden szinten néhány száz bájt, mert egy körfüggvényből áll, amelyet a kulcsgenerálás is használ. Néhány tíz kilobájtos flash mellett (2.3.1. alfejezet) a kódméret önmagában is kizárhat egy változatot.

A 5.5. ábra a kódméretet és a fajlagos költséget együtt mutatja az optimalizált szinteken. A SPECK mindkét szempontból a legjobb, nála nem kell választani méret és sebesség között. A LEA ugyanolyan gyors, de a kibontott KISA-kód miatt kb. 36-szor akkora. Az AES és az Ascon-AEAD128 nagyjából azonos méretű és sebességű, az AES a táblák, az Ascon a kibontott permutáció miatt. Az Ascon-AEAD128-nál a méretre optimalizálás egyik szempontból sem előnyös: `-Os` mellett a kódja nagyobb, és 15 %-kal lassabb is.



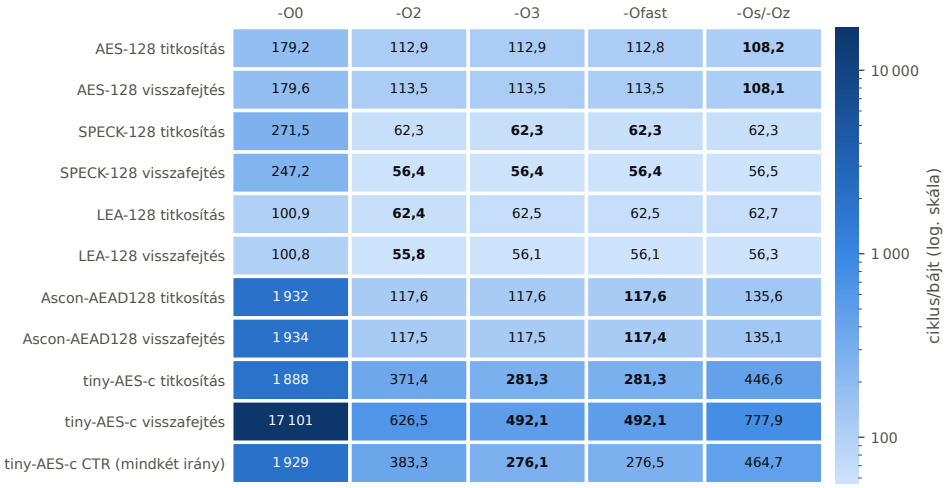

5.5. ábra. Flash-igény és fajlagos titkosítási költség (1024 B) az optimalizált szinteken, mindkét tengely logaritmikus. Az `-Os` pont az `-Oz`-t is jelenti; a szürke pontok a `tiny-AES-c`-t mutatják.

## 5.6. A fordítói optimalizáció hatása

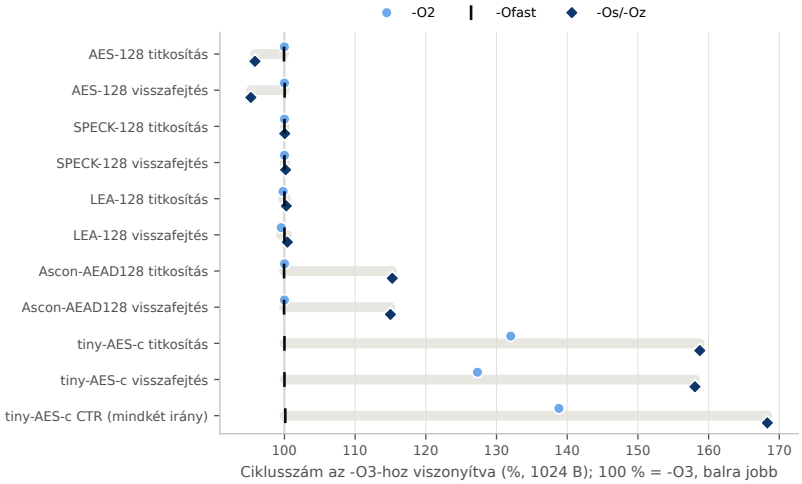
Az 5.6. ábra minden algoritmus és minden szint fajlagos költségét mutatja, a 5.7. ábra pedig az `-O3`-hoz viszonyított eltérést.

### 5.6.1. Optimalizálás nélkül és optimalizálva

Az optimalizálás az algoritmusokat nagyon eltérő mértékben gyorsította. 1024 bájtos titkosításnál `-O0`-hoz képest `-O3` mellett az Ascon-AEAD128 16,4-szer, a SPECK 4,4-szer, a LEA 1,6-szer (visszafejtésben 1,8-szer), az AES szintén 1,6-szer gyorsabb. A T-táblás AES-kör és a kibontott KISA-féle LEA-kör már a forrásban is egyszerű műveletek so-




5.6. ábra. Fajlagos költség (ciklus/bájt) 1024 bájtos üzeneten, minden optimalizációs szinten. Soronként félkövér a legjobb érték; az alsó három sor a `tiny-AES-c`.




5.7. ábra. A ciklusszám az `-O3`-on mért érték százalékában (1024 B); 100 % alatt az adott szint gyorsabb. Az alsó három sor a `tiny-AES-c`.

rozata, függvényhívás nélkül, így a fordítónak kevés kiküszöbölhető többlet marad. A SPECK 64 bites szavait viszont 32 bites regiszterpárokban kell kezelni, és optimalizálás nélkül minden részeredmény a verembe kerül. Az Ascon-AEAD128 lassúságát `-O0` mellett a kényszerített beágyazás okozza (5.5. alfejezet). Ennek eredményeként `-O0` mellett a LEA a leggyorsabb (100,9 CPB, az AES 179,2, a SPECK 271,5), az Ascon-AEAD128 pedig 1 932 CPB-vel a mezőny végén van, 16 bájton 41-szer drágább az AES-nél, optimalizálva csak 3,7-szer. Az `-O0` tehát az algoritmusok sorrendjét is torzítja, ezért fejlesztői beállítással mért adatokból nem szabad következtetni a valódi költségre.

### 5.6.2. `-O2`, `-O3` és `-Ofast`

A négy algoritmus titkosítása és visszafejtése már `-O2` mellett eléri a sebességre optimalizált szintek legjobb értékét, az `-O3` és az `-Ofast` egyiknél sem javít rajta (a LEA `-O2` mellett 0,2–0,5 %-kal még gyorsabb is). Az AES-nél az `-O3` csak a titkosító kulcsgenerálást gyorsítja (5.1. alfejezet). Az `-Ofast` a titkosítás és a visszafejtés értékeiben legfeljebb 0,5 %-kal tért el az `-O3`-tól. Ez várható volt, mert az `-Ofast` többlete a lebegőpontos számításokat érinti, a vizsgált algoritmusok pedig csak egész műveleteket használnak.

### 5.6.3. `-Os/-Oz`

A méretre optimalizált kód hatása algoritmusonként eltér. Az AES `-Os/-Oz` mellett 4–5 %-kal gyorsabb, mint `-O3` mellett (108,2, illetve 108,1 CPB), és a kódja is itt a legkisebb. A SPECK és a LEA gyakorlatilag érzéketlen a beállításra (0,5 % alatti eltérés). Az Ascon-AEAD128 viszont mindkét irányban 15 %-kal lassabb, mint `-O3` mellett.

Az Ascon-AEAD128 lassulását a generált kód magyarázza. A `P8` függvény `-O3` mellett 489 utasítás, és mindössze 3 veremhozzáférést tartalmaz. `-Os` mellett 659 utasítás, ebből 173 veremhozzáférés, mert a fordító a regiszterekbe nem férő állapotszavakat a verembe menti, majd onnan olvassa vissza. A `P12`-nél ugyanez a helyzet (720, illetve 963 utasítás, 2, illetve 243 veremhozzáféréssel), és mindkét függvény kódja 17–20 %-kal hosszabb lesz.

Az AES-nél inkább az adat-gyorsítótár mérete számíthat: a 10 KB-os táblák nem férnek el az ART 256 bájtos adat-gyorsítótárában, így a táblaolvasások egy része várakozási ciklusokkal jár. A táblák SRAM-ba helyezése ezért valószínűleg gyorsítaná az AES-t; ezt nem vizsgáltuk.

### 5.6.4. A LEA és az utasítás-gyorsítótár

A LEA két iránya közötti különbség és az, hogy a LEA nem gyorsabb a SPECK-nél, a generált gépi kódból érthető meg. A `lea.c`-t a GCC 14.2.1-es változatával külön lefordítottuk (a kód mérete 4 bájton belül egyezett a mért buildekével), és az `arm-none-eabi-objdump` kimenetén megszámoltuk, mi fut le egy blokkon 128 bites kulccsal.

A két irány kódja szerkezetében tükörképe egymásnak. A fordító mindkettőből körönként 15 utasítást készít: hat körkulcsszó-betöltést, hat XOR-t és három összeadást, illetve kivonást. A forgatásokat a Cortex-M4 eltolóegysége egy másik utasításba olvasztja: titkosításkor a következő XOR operandusába (például `eor.w r3, r3, r4, ror #3`), visszafejtéskor a kivonásba (például `rsb r6, r6, r3, ror #9`). Veremmentés egyik irányban sincs. A végrehajtott utasítások száma így titkosításkor 373, visszafejtéskor 370, ami 1 %-on belül egyezik, a mért költség viszont 11 %-kal eltér (blokkonként 1 000, il-

letve 897 ciklus). Az eltérés a kód hosszával arányos: a titkosító út 1 268, a visszafejtő 1 132 bájt, mert titkosításkor a forgatott operandusú XOR-ok miatt több a 32 bites utasítás. A két arány (1,12 és 1,115) jól egyezik, és mindkét irányban kb. 0,78 ciklus jut a végrehajtott kód egy bájtjára.

A magyarázat az utasítás-gyorsítótár mérete. A kibontott 24 kör egyik irányban sem fér el az ART 1 KB-os utasítás-gyorsítótárában, így minden blokknál végigfutva kiszorítja a saját korábbi sorait, és az utasításokat minden alkalommal újra a flash-ből kell olvasni, 4 várakozási ciklussal és előolvasás nélkül. Ilyenkor egy 8 bájtos flash-sor beolvasása 5 ciklus, vagyis bájtonként 0,625 ciklus, ami a mért 0,78-hoz képest jó alsó becslés. A futási időt tehát itt elsősorban a kód flash-ből olvasása határozza meg, nem az utasítások száma. Ugyanez igaz az Ascon-AEAD128 1,8–2,1 KB-os `P8` függvényére és a LEA kulcsgenerálására is (5.1. alfejezet). A SPECK titkosító ciklusa ezzel szemben 64 bájt, elfér a gyorsítótárban.

Ezzel a 3.3. alfejezetben nyitva hagyott kérdés is megválaszolható. A tervezők szerint 32 bites processzoron a LEA a gyorsabb, a SPECK 128/128 csak 64 bites gépen előzi meg [15]. Méréseink ezt nem igazolták: optimalizált kódban a két algoritmus fajlagos költsége 1 %-on belül azonos. A LEA körönként valóban kevesebb utasítást igényel (15, a SPECK titkosításkor 21), de a teljesen kibontott KISA-kód a flash várakozási ciklusai miatt nem tudja ezt kihasználni, a SPECK rövid ciklusa viszont a gyorsítótárból fut. A LEA 32 bites szóméretből fakadó előnye tehát ezen a platformon a memória-hozzáférésen vész el. Előolvasással, a kód SRAM-ba helyezésével vagy ciklusos, rövidebb megvalósítással ez megváltozhat; ezt nem mértük (7.6. alfejezet).

## 5.7. Összevetés a `tiny-AES-c` könyvtárral

A `tiny-AES-c` könyvtárat [35] azért mértük a referenciakóddal azonos keretben, mert jól mutatja, mennyire függ az eredmény az implementációtól (5.5. táblázat).

5.5. táblázat. A tervezők AES-referenciakódja és a `tiny-AES-c` (1024 B, ciklus/bájt)

| **Mért érték** | `-O0` | `-O3` | `-Os/-Oz` |
|---|---|---|---|
| Referenciakód, titkosítás | 179,2 | 112,9 | 108,2 |
| Referenciakód, visszafejtés | 179,6 | 113,5 | 108,1 |
| `tiny-AES-c`, titkosítás (ECB) | 1 887,6 | 281,3 | 446,6 |
| `tiny-AES-c`, visszafejtés (ECB) | 17 101,0 | 492,1 | 777,9 |
| `tiny-AES-c`, CTR (mindkét irány) | 1 928,7 | 276,1 | 464,7 |
| Lassulás a referenciakódhoz képest (T / V) | 10,5 / 95,2 | 2,5 / 4,3 | 4,1 / 7,2 |

A `tiny-AES-c` kis kódméretre és hordozhatóságra készült: csak a 256 bájtos S-boxot és inverzét tárolja, bájtonként dolgozik, a $GF(2^8)$-szorzást pedig futás közben számolja. A MixColumns oszloponként 4 `xtime` hívással (kettővel való szorzással) dolgozik. Az inverz MixColumns viszont egy általános `Multiply` makróval szoroz, amely az együttható-

tól függetlenül mindig kiszámolja a teljes, 10 `xtime` hívásból álló láncot, és oszloponként 16 ilyen szorzást végez. Egy blokk titkosítása így 144, visszafejtése 5 760 `xtime` hívást tartalmaz, negyvenszer annyit. Optimalizálás nélkül minden ilyen hívás valódi függvényhívás, ezért a visszafejtés 9,1-szer drágább a titkosításnál, és 95-ször a referenciakódnál. `-O3` mellett a fordító a hívásokat beágyazza, és a konstans együtthatók miatt a felesleges tagok nagy részét kiejti, a visszafejtés azonban így is 1,75-ször drágább. `-Os`/`-Oz` mellett a segédfüggvények nem ágyazódnak be, ezért a könyvtár itt a leglassabb, miközben a referenciakód éppen itt a leggyorsabb. A CTR mód mindkét iránya a titkosító függvényt hívja, ezért ott nincs ilyen aszimmetria.

Ezért a `tiny-AES-c` ebben az összehasonlításban torz viszonyítási alap lenne: a SPECK, a LEA és az Ascon referenciakódjával szemben az algoritmusok helyett eltérő célú implementációkat mérnénk. `-O3` mellett a `tiny-AES-c`-hez képest a SPECK és a LEA titkosításkor 4,5-ször, visszafejtéskor 8,7–8,8-szor gyorsabbnak látszana, a referenciakódhoz képest csak 1,8-szor, illetve 2,0-szer gyorsabb. Az Ascon-AEAD128 a `tiny-AES-c`-nél kb. 29 bájt (`-Os`/`-Oz` mellett kb. 18 bájt) fölött gyorsabb, 1024 bájton 2,4-szer (5.4. ábra), a csak titkosító referencia-AES-nél viszont egyik mért méreten sem. A `tiny-AES-c`-vel mérve tehát a K2 kérdésre is mást válaszolnánk: az Ascon a hitelesítéssel együtt is jóval gyorsabbnak látszana az AES-nél.

A `tiny-AES-c` valódi előnye a kódméret: az ECB- és a CTR-móddal együtt 1,5–5,1 KB a referenciakód 12,8–14,6 KB-jával szemben. Nagyon szűkös flash mellett ez dönthet, de ilyenkor is 2,5–7-szeres lassulással kell számolni, és a SPECK kisebb kóddal is gyorsabb. Konstans idejű egyik AES-megvalósítás sem.

## 5.8. Összevetés a szakirodalmi eredményekkel

A mért értékeket érdemes optimalizált megvalósításokéhoz is mérni; az alábbi eredmények más kártyán, más feltételek mellett születtek, ezért csak tájékoztató jellegűek. Schwabe és Stoffelen Cortex-M4-re assemblyben írt AES-128-CTR implementációja táblás változatban blokkonként kb. 554 ciklust (kb. 35 ciklus/bájt), a gyorsítótár-alapú támadások ellen védett, bitsliced változatban kb. 1 618 ciklust (kb. 101 ciklus/bájt) igényel [44]. Az általunk mért C-referenciakód 108,1–112,9 ciklus/bájttal a táblás assembly-változatnál kb. háromszor, a bitsliced változatnál 7–12 %-kal lassabb. A különbséghez a kézzel írt assembly mellett valószínűleg az is hozzájárul, hogy nálunk a táblák a flash-memóriában vannak (5.6. alfejezet). A LEA tervezői optimalizált kóddal ARM926EJ-S processzoron 20 ciklus/bájtot közöltek [15], a KISA hordozható C-kódja nálunk az optimalizált szinteken 55,8–62,7 ciklus/bájtot ért el.

Az AES-hez viszonyított arányaink tehát a hordozható C-kódra érvényesek: a C-ben írt SPECK és LEA (56–63 ciklus/bájt) a bitsliced assembly-AES-nél is gyorsabb, a táblás assembly-AES-nél viszont nem.

## 5.9. Az eredmények összegzése

`-O3` mellett a SPECK és a LEA fajlagos költsége gyakorlatilag azonos, és kb. fele a T-táblás AES-énak. Az Ascon-AEAD128 bájtonkénti költsége megegyezik az AES-évél, így hosszú üzeneten a hitelesítéssel együtt is csak 4 %-kal drágább, rövid üzeneteken viszont a kb. 4 900 ciklusos fix költsége a hátránya. A legjobb fordítási szint algoritmusfüggő, ezért ezt is mérés alapján érdemes megválasztani. Az eredményeket részletesen a 7.3. alfejezet foglalja össze.

# 6. fejezet

# Biztonsági elemzés és összehasonlító értékelés

A teljesítmény mellett legalább ilyen fontos, hogy egy algoritmus milyen védelmet nyújt az ismert támadásokkal szemben. Ez a fejezet a szakirodalom alapján foglalja össze a négy algoritmus biztonsági jellemzőit; támadásokat nem valósítottunk meg.

## 6.1. Alapfogalmak

### 6.1.1. Támadási modellek

A támadó képességei szerint négy, egyre erősebb modellt szokás megkülönböztetni [1]. A *csak titkosított szöveget ismerő* (ciphertext-only) támadó kizárólag a titkosított kimenetet látja. Az *ismert nyílt szöveges* (known-plaintext) modellben nyílt és titkosított szövegpárok is rendelkezésére állnak. A *választott nyílt szöveges* (chosen-plaintext, CPA) modellben maga választhatja meg a titkosítandó üzeneteket; ez reálisabb, mint elsőre tűnik, mert sok protokollban a támadó rá tudja venni az eszközt bizonyos üzenetek titkosítására. A *választott titkosított szöveges* (chosen-ciphertext, CCA) modellben a visszafejtendő szövegeket is ő választja. A modern algoritmusoktól általában a CPA- vagy a CCA-modellben várnak el biztonságot.

### 6.1.2. Biztonsági szint és biztonsági tartalék

Egy algoritmus *biztonsági szintje s* bit, ha a legjobb ismert támadás $2^s$ műveletet igényel. Ideális esetben ez a kulcs hossza, vagyis a legjobb támadás a kulcstér végigpróbálása, ami 128 bites kulcsnál $2^{128} \approx 3{,}4 \times 10^{38}$ művelet.

A *biztonsági tartalék* (security margin) azt fejezi ki, hogy az algoritmus teljes körszáma ($N$) mennyivel több a legjobb ismert támadás által elért körszámnál ($n$): $N - n$ kör, vagy arányosan $(N-n)/N$. Minél nagyobb a tartalék, annál több tér marad a jövőbeli,

erősebb támadások számára [7]. Eltérő körszámú algoritmusokat csak az arányos értékkel lehet összevetni.

## 6.2. A főbb támadástípusok

### 6.2.1. Differenciális kriptoanalízis

A differenciális kriptoanalízist Biham és Shamir vezette be 1991-ben a DES elemzésére [26]. A támadó olyan nyíltszöveg-párokat választ, amelyek között ismert (általában XOR-) különbség van, és azt vizsgálja, hogyan terjed ez a különbség a körökön át. Ha bizonyos bemeneti különbségek a véletlennél nagyobb valószínűséggel adnak bizonyos kimeneti különbségeket, abból a körkulcsok egyes bitjeire lehet következtetni. A tervezők ezért úgy választják meg az S-boxokat és a körszámot, hogy a teljes titkosításon átívelő differenciális jellemzők valószínűsége elhanyagolható legyen. ARX-algoritmusoknál az elemzés nehezebb, mert a moduláris összeadás differenciális viselkedése az átvitelek miatt bonyolultabb, mint egy S-boxé.

### 6.2.2. Lineáris kriptoanalízis

Matsui 1993-ban publikálta a lineáris kriptoanalízist, szintén a DES-re [27]. A támadó a nyílt szöveg, a titkosított szöveg és a kulcs bitjei közötti olyan lineáris (XOR-) összefüggéseket keres, amelyek a véletlennél gyakrabban teljesülnek; elég sok ismert szövegpárból ezekkel kulcsbitek nyerhetők ki. Bogdanov és Rijmen később bevezették a nulla korrelációjú lineáris kriptoanalízist [20], a differenciális és a lineáris módszer kombinációja, a differenciális-lineáris támadás pedig az ARX-algoritmusok ellen az egyik leghatékonyabbnak bizonyult [22].

### 6.2.3. Algebrai és rotációs támadások

Az algebrai támadások az algoritmust $GF(2)$ feletti egyenletrendszerként írják fel, és ennek megoldásával próbálják meghatározni a kulcsot. A teljes körszámú algoritmusok egyenletrendszere azonban túl nagy a mai megoldók (Gröbner-bázisok, SAT-megoldók) számára, az ARX-algoritmusoknál pedig a moduláris összeadás bitszintű leírása még bonyolultabb egyenletekhez vezet [7].

A Khovratovich és Nikolić által bevezetett rotációs kriptoanalízis [28] kifejezetten ARX-szerkezetek ellen készült. Azt használja ki, hogy elforgatott bemenetre egy ideális ARX-rendszer kimenete is elforgatva jelenne meg; ezt az összefüggést a moduláris összeadás és a körkonstansok megtörik, a támadás pedig azt vizsgálja, mennyire. A SPECK és a LEA teljes változata ellen ez a módszer sem vezetett eredményre.

### 6.2.4. Oldalcsatorna-támadások

Az oldalcsatorna-támadások nem az algoritmus matematikai szerkezetét, hanem a fizikai megvalósítást használják ki [29]: a végrehajtási idő, az áramfelvétel (egyszerű és differenciális teljesítményanalízis, SPA és DPA), az elektromágneses kisugárzás vagy a gyorsítótár-hozzáférések mintázata információt hordozhat a kulcsról.

Az *időzítéses* és *gyorsítótár-alapú* támadások ellen a konstans idejű kód véd. A táblás AES-megvalósításokban, így az általunk mért T-táblás referenciakódban is, a táblaolvasás címe a kulcstól és az adattól függ, ezért gyorsítótárral rendelkező rendszereken a hozzáférési idő árulkodó lehet [34]. A vizsgált mikrovezérlőn az ART adat-gyorsítótára a flash elé kapcsolódik, így ez a veszély elvben itt is fennáll. Az ARX-algoritmusok és az Ascon bitsliced S-boxa nem használ adatfüggő memóriacímet vagy elágazást, ezért természetüknél fogva konstans idejű. A *teljesítményanalízis* ellen viszont a konstans idejűség nem véd: ehhez maszkolás (a köztes értékek véletlen részekre bontása) szükséges. A maszkolás az alacsony fokú S-boxot használó Asconnál viszonylag olcsó, ez a tervezők kifejezett célja volt [9]; az ARX-algoritmusoknál a moduláris összeadás, az AES-nél a nagy S-box miatt drágább.

Oldalcsatorna-méréseket nem végeztünk, mert ehhez oszcilloszkóp és speciális mérőeszközök szükségesek; ez a szakasz a szakirodalmi áttekintés része.

### 6.2.5. Születésnapi korlát

A születésnapi paradoxon miatt $n$ bites blokkméret mellett nagyjából $2^{n/2}$ blokk feldolgozása után nagy valószínűséggel ismétlődik egy titkosított blokk. CBC módban például két titkosított blokk egyezéséből a megfelelő nyíltszöveg-blokkok XOR-ja kiszámítható, és a CBC-MAC is hamisíthatóvá válik. A vizsgált blokktitkosítók 128 bites blokkmérete mellett ez a korlát $2^{64}$ blokk, ami IoT-alkalmazásokban gyakorlatilag elérhetetlen. Ezért vizsgáltuk a 64 bites blokkméretű változatok helyett a 128 biteseket.

## 6.3. Az algoritmusok biztonsági jellemzői

### 6.3.1. AES-128

Az AES-128 a legalaposabban vizsgált blokktitkosítók egyike. A legjobb ismert kulcsvisszafejtő támadások, például a meet-in-the-middle támadások, a 10 körből 7-et érnek el [41], ami 3 kör, azaz 30 % tartalék. A teljes algoritmus ellen a legjobb ismert eljárás a biclique-támadás, amely a kulcstér végigpróbálásához képest mindössze kb. négyszeres gyorsulást ér el ($2^{126,1}$ művelet), így gyakorlati jelentősége nincs [19].

### 6.3.2. SPECK 128/128

A SPECK 128/128 ellen a legjobb publikált támadás differenciális, és a 32 körből 23-at ér el, közel a kulcstér végigpróbálásának költségével [21]. Ez 9 kör, azaz kb. 28 % tartalék; a tervezők a körszámot kifejezetten úgy választották meg, hogy a tartalék az AES-éhez hasonló legyen [13]. Rotációs támadás a teljes változat ellen nem ismert.

### 6.3.3. LEA-128

A LEA-128 ellen a legjobb ismert támadás differenciális-lineáris, és a 24 körből 17-et ér el [22]. Ez 7 kör, azaz kb. 29 % tartalék. A tervezők részletes biztonsági elemzést közöltek a forgatási paraméterek megválasztásáról és a differenciális, illetve lineáris tulajdonságokról [15], az algoritmus pedig ISO/IEC-szabvány része [16].

### 6.3.4. Ascon-AEAD128

Az Ascon biztonságát AEAD-sémaként kell értékelni. A tervezők a titkosságra és a hitelességre egyaránt 128 bites biztonságot állítanak. A leghosszabb ismert kulcsvisszafejtő támadások a 12 körös inicializálásból 7 kört érnek el, ami 5 kör, azaz kb. 42 % tartalék [9, 43]. Rohit és munkatársai nonce-ismétlést nem igénylő 7 körös támadása még kb. $2^{123}$ műveletet igényelt [43]; ezt későbbi munkák kb. $2^{96}$ műveletre csökkentették $2^{64}$ adat mellett, a körszám azonban továbbra is 7 [46]. Ezek a támadások az inicializálást célozzák, amely a v1.2-es változatokban és az Ascon-AEAD128-ban ugyanaz a 12 körös permutáció (csak a kezdőérték és a bájtsorrend tér el), ezért a szabványos sémára is érvényesek. Megvalósítási szempontból előnyös, hogy konstans idejű, és viszonylag olcsón maszkolható.

Mint minden nonce-alapú AEAD-sémánál, az Asconnál is kritikus, hogy ugyanazzal a kulccsal egy nonce ne ismétlődjön: azonos kulcs, nonce és kapcsolódó adat mellett a két titkosítás ugyanabból az állapotból indul, ezért az első blokkok (és az azonos előtag utáni első eltérő blokk) nyílt szövegeinek XOR-ja kiolvasható a titkosított szövegekből. Az első eltérő blokk után az állapotok szétválnak, a további blokkokra ez már nem érvényes, a titkossági és hitelességi garanciák viszont így is elvesznek. Ez a séma használati feltétele, amelyet az alkalmazásban (például számlálóalapú nonce-szal) kell biztosítani; a mérésekben használt csupa nulla nonce csak mérési célt szolgált.

## 6.4. Összehasonlító értékelés

A 6.1. táblázat a fenti jellemzőket foglalja össze.

A három blokktitkosító arányos biztonsági tartaléka hasonló (28–30 %), az Ascon-

6.1. táblázat. A vizsgált algoritmusok biztonsági jellemzői a szakirodalom alapján

| **Szempont** | **AES-128** | **SPECK 128/128** | **LEA-128** | **Ascon-AEAD128** |
|---|---|---|---|---|
| Állított biztonság | 128 bit | 128 bit | 128 bit | 128 bit |
| Legjobb támadás (kör) | 7 / 10 | 23 / 32 | 17 / 24 | 7 / 12[a] |
| Támadás típusa | MITM | differenciális | diff.-lineáris | cube-típusú[a] |
| Biztonsági tartalék | 30 % | 28 % | 29 % | 42 % |
| Konstans idejű kód | nehéz[b] | természetes | természetes | természetes |
| Maszkolás költsége | nagy | nagy | nagy | kisebb |
| Szabvány | FIPS 197 | ISO/IEC 29167-22[c] | ISO/IEC 29192-2 | NIST SP 800-232 |

[a]Az inicializáló permutáció 12 köréből.
[b]A mért T-táblás megvalósítás nem konstans idejű; bitsliced szoftveres vagy hardveres megvalósítással megoldható.
[c]Csak RFID-légi interfészhez.
A támadások modellje, adat- és időigénye eltér, ezért a tartalékok csak tájékoztató jellegűek.

AEAD128 inicializálásáé ennél nagyobb. Ebből csak annyi következik, hogy a publikált támadások egyik blokktitkosító teljes körszámához sem jutottak lényegesen közelebb, mint a többiéhez. Azonos biztonsági szintet ez nem bizonyít. A támadások modellje, adat- és időigénye eltér: a SPECK 23 körös támadása például kb. $2^{125}$ választott nyílt szöveget igényel [21], az AES 7 körös támadásai viszont $2^{100}$ alatti adattal és idővel dolgoznak [41]. Az AES-t ráadásul jóval régebben és többen elemzik, mint a két ARX-algoritmust. A gyakorlati különbségek inkább a megvalósításban jelentkeznek. Az ARX-algoritmusok és az Ascon konstans idejű kódja egyszerűbb, mint a táblás AES-é, a teljesítményanalízis elleni maszkolás pedig az Asconnál a legolcsóbb. A SPECK hátránya nem kriptoanalitikai, hanem elfogadottsági: az általános célú könnyűsúlyú blokktitkosító-szabványba nem került be, és tervezésének körülményei vitatottak.

# 7. fejezet

# Összefoglalás és továbbfejlesztési lehetőségek

A dolgozat célja négy, eltérő felépítésű kriptográfiai algoritmus – az AES-128, a SPECK 128/128, a LEA-128 és az Ascon-AEAD128 – futásidejű költségének összevetése volt egy valós, ARM Cortex-M4 magos mikrovezérlőn (STM32L432KC), a kódméret és a veremigény vizsgálatával és a biztonsági jellemzők szakirodalmi áttekintésével kiegészítve.

## 7.1. Felhasznált módszerek és eszközök

A mérések egy NUCLEO-L432KC fejlesztőkártyán készültek, 80 MHz-en. A programot STM32CubeIDE-ben, GCC-vel hat optimalizációs szinten fordítottuk. A futási időt a DWT ciklusszámlálóval mértük, tiltott megszakítások mellett, százszoros ismétléssel és tíz kör mediánjával; az eredményeket soros porton gyűjtöttük, és Python-szkriptekkel (matplotlib) dolgoztuk fel. A kódméretet a linker térképfájljából, a veremigényt a GCC hívási gráfjából (`-fcallgraph-info=su`) határoztuk meg. A LEA, a SPECK és az Ascon-AEAD128 futási idejének okait a generált gépi kód (`arm-none-eabi-objdump`) elemzésével vizsgáltuk. A biztonsági értékelés a szakirodalomban közölt kriptoanalitikai eredményeken alapul.

## 7.2. A saját munka

A háttérismeretek (2. fejezet), az algoritmusok leírása (3. fejezet) és a biztonsági jellemzők (6. fejezet) a hivatkozott szakirodalmon alapulnak. A mért algoritmuskódok nyilvános referenciakódok: az AES-hez a tervezők kódját [33], összevetésként a `tiny-AES-c` könyvtárat [35], a SPECK-hez az NSA megvalósítási útmutatójának kódját [14], a LEA-hoz a KISA hivatalos forráskódját [31], az Ascon-AEAD128-hoz a tervezők `ascon-c` könyvtárát [36] használtuk fel. A SPECK-nél az útmutató kódját kisebb módosítá-

sokkal (`const` paraméterek, a visszafejtés paramétersorrendje, `memcpy`-alapú bájtátalakítás) vettük át. Az AES, a LEA és az Ascon-AEAD128 referenciakódját az algoritmusrész módosítása nélkül illesztettük be egy-egy `.c`/`.h` fájlpárba; ehhez csak a fejlécfájlhivatkozásokat, a típusdefiníciókat és egy platformspecifikus paraméterellenőrző makrót kellett igazítani. A mérőkeret, vagyis az ECB-ciklus, a ciklusszámláló és az önellenőrzés, saját munka. Ezen kívül a saját munka a következőkre terjedt ki:

– a referenciakódok kiválasztása és beillesztése (a KISA LEA-csomagjából és az `ascon-c` könyvtárból a szükséges részek összefűzése, a `tiny-AES-c` szűkítése 128 bites kulcsra), a helyesség ellenőrzése hivatalos tesztvektorokkal;

– a mérések elvégzése hat optimalizációs szinten és négy üzenetméreten, a kódméret és a veremigény meghatározása, a kiértékelő szkriptek, ábrák és táblázatok elkészítése;

– az eredmények értelmezése (a két AES-megvalósítás eltérésének oka, az Ascon-AEAD128 költségmodellje és összevetése egy AES-CCM becsült költségével, a LEA, a SPECK és az Ascon-AEAD128 futási idejének visszavezetése a generált kódra és az utasítás-gyorsítótár méretére, az összevetés a szakirodalmi eredményekkel) és az alkalmazási ajánlások.

## 7.3. Az eredmények tézisszerű összefoglalása

A tézisek után zárójelben áll, hogy melyik bevezetőben feltett kérdésre (K1–K4) adnak választ.

1. **Az ARX-alapú algoritmusok fajlagos költsége kb. fele az AES-énak (K1).** `-O3` mellett a SPECK és a LEA titkosításkor 62, visszafejtéskor 56 ciklust igényelt bájtonként, a tervezők T-táblás AES-referenciakódja 113-at; ez titkosításkor 1,8-szoros, visszafejtéskor 2,0-szeres különbség. A SPECK kódja optimalizált szinteken 300 bájt alatti, a LEA-é a teljesen kibontott KISA-kód miatt kb. 10,7 KB, az AES-é a táblák miatt 12,8–14,2 KB.

2. **Az AES-eredmény erősen implementációfüggő (K1, K3).** A kis kódméretre írt `tiny-AES-c` optimalizált fordítással 2,5–7,2-szer lassabb volt a referenciakódnál, mert a $GF(2^8)$-szorzást futás közben, bájtonként számolja; vele mérve a SPECK és a LEA előnye 2,5–4,3-szor akkorának látszott volna. A referenciakódban a két irány költsége azonos, mert az inverz MixColumns munkája a visszafejtő kulcs előállításába kerül.

3. **Az Ascon-AEAD128 hosszú üzeneteken alig drágább a csak titkosító AES-nél, egy hitelesített AES-módnál pedig olcsóbb (K2).** Költsége `-O3` mellett egy 4 926

ciklusos fix tag (a két 12 körös permutáció) és 16 bájtonként 1 805 ciklus összege, bájtonkénti költsége tehát megegyezik az AES-ével. Így 16 bájton 3,7-szer, 1024 bájton csak 1,04-szer drágább az ECB módú AES-nél, holott a hitelesítést is elvégzi. Egy AES-CCM becsült költségénél már 16 bájton olcsóbb, 1024 bájton kb. feleannyiba kerül, és nincs szüksége külön kulcsgenerálásra.

4. **A fordítási beállítás hatása nagy és algoritmusfüggő (K3).** Az optimalizálás az Ascon-AEAD128-at 16,4-, a SPECK-et 4,4-, a LEA-t és az AES-t 1,6–1,8-szeresére gyorsította, így `-O0` mellett a LEA a leggyorsabb, az Ascon-AEAD128 pedig a kényszerített beágyazás miatt a leglassabb, 161 KB-os kóddal. Az `-Ofast` legfeljebb 0,5 %-kal tért el az `-O3`-tól, az `-Os` és az `-Oz` azonos kódot adott. Méretre optimalizálva az AES 4–5 %-kal gyorsabb, az Ascon-AEAD128 15 %-kal lassabb lett, mert a fordító a permutáció állapotának egy részét a veremben tartja.

5. **A LEA 32 bites előnye ezen a platformon nem jelent meg (K1, K3).** A LEA körönként 15, a SPECK 21 utasítást igényel, a mért költségük mégis 1 %-on belül azonos. A kibontott KISA-kód nem fér el az 1 KB-os utasítás-gyorsítótárban, futási ideje a végrehajtott kód hosszával arányos.

6. **A publikált támadások nem jeleznek gyengeséget (K4).** A szakirodalom alapján a három blokktitkosító arányos biztonsági tartaléka hasonló (28–30 %), és egyik ellen sem ismert gyakorlatban kivitelezhető támadás. Ez nem jelent bizonyítottan azonos biztonsági szintet, mert a támadások feltételei eltérnek, de nem utal arra, hogy a könnyűsúlyú blokktitkosítók teljesítményelőnye a biztonság rovására menne.

## 7.4. Alkalmazási ajánlások

A 7.1. táblázat a mérések és a biztonsági elemzés alapján tipikus helyzetekre fogalmaz meg ajánlásokat. Az ajánlások 32 bites Cortex-M4 platformon mért adatokon alapulnak, más architektúrákra csak óvatosan vihetők át.

A SPECK és a LEA önmagában csak titkosságot nyújt; ha hitelesség is kell, külön MAC-ra vagy hitelesített üzemmódra van szükség, ami a költségüket növeli.

7.1. táblázat. Alkalmazási ajánlások 32 bites mikrovezérlőkre

| Helyzet | Ajánlás | Indoklás |
|---|---|---|
| Titkosság és hitelesség is szükséges | Ascon-AEAD128 | Szabványos, egy sémában adja a titkosítást és a hitelesítést, külön kulcsgenerálás nélkül, konstans idejű kóddal. Hosszú üzeneteken alig drágább a csak titkosító AES-nél, egy becsült AES-CCM-nél már 16 bájton olcsóbb. Sebességre optimalizálva (`-O2`) érdemes fordítani. |
| Nagy feldolgozási sebesség, a hitelesítést más réteg biztosítja | SPECK 128/128 vagy LEA-128 | A legnagyobb feldolgozási sebesség (1 280–1 430 kB/s), kb. kétszerese az AES-ének. A SPECK kódja 300 bájt alatti, a LEA ISO/IEC-szabvány, de a KISA-kódja kb. 10,7 KB. |
| Nagyon rövid üzenetek, gyakori kulcscsere | SPECK 128/128 | Nincs üzenetenkénti fix költség, és a kulcsgenerálás olcsó (kb. 800 ciklus). A LEA kulcsgenerálása kb. 1 300, az AES titkosító kulcsáé 500–630, a visszafejtőé kb. 2 700–3 100 ciklus. |
| Szabványmegfelelés (pl. FIPS) kötelező vagy van hardveres AES-gyorsító | AES-128 | Hardveres gyorsítóval a szoftveres hátrány megszűnik. Szoftveresen a T-táblás megvalósítás kb. kétszer lassabb az ARX-algoritmusoknál, kb. 10 KB táblát igényel, és nem konstans idejű. CTR-re épülő módban (GCM, CCM) a visszafejtő táblák elhagyhatók. |

## 7.5. Korlátok

– A mérések egyetlen platformon (STM32L432KC, Cortex-M4) készültek; más architektúrákon, különösen 8 és 16 bites eszközökön, az arányok eltérhetnek.

– Mind a négy algoritmusnál a hordozható C-referenciakódot mértük, a kód és az AES-táblák a flash-memóriában voltak. Assembly-változatokkal, SRAM-ba helyezett kóddal vagy táblákkal, illetve bekapcsolt előolvasással az arányok eltérhetnek; a LEA-nál és az Ascon-AEAD128-nál valószínűleg jelentősen.

– A blokktitkosítókat ECB módban, az Ascon-AEAD128-at AEAD-ként mértük, így az utóbbi költsége a hitelesítést is tartalmazza; hitelesített AES-módot (GCM, CCM) nem mértünk, a CCM költségére csak becslést adtunk.

– Csak blokkhatárra eső üzenetméreteket (16, 64, 256, 1024 bájt) mértünk, a részleges blokkok kezelésének költségét nem, és az Ascon-AEAD128-at csak üres kapcsolódó adattal.

– Az utasítás-gyorsítótárra vonatkozó magyarázatot a generált kód elemzésére alapoztuk, külön méréssel nem ellenőriztük.

– Egyetlen fordítóval (GCC 14.3.1) mértünk; más fordító vagy fordítóverzió más arányokat adhat, az Ascon-AEAD128 `-Os` melletti veremmentései (5.6. alfejezet) szerint akár a regiszterkiosztás miatt is.

– Energiafogyasztást és oldalcsatorna-szivárgást nem mértünk; a biztonsági értékelés szakirodalmi adatokon alapul.

## 7.6. Továbbfejlesztési lehetőségek

**A flash-memória és a gyorsítótár hatásának mérése.** Az ART előolvasásának bekapcsolásával, az utasítás-gyorsítótár kikapcsolásával, illetve a LEA és az Ascon-AEAD128 forró kódjának SRAM-ba helyezésével közvetlenül is ellenőrizhető lenne, mennyit veszítenek a flash várakozási ciklusai miatt. Ugyanez az AES-tábláknál is megmutatná az adat-gyorsítótár szerepét.

**Optimalizált implementációk.** Cortex-M-re optimalizált assembly-változatok (például az `ascon-c armv7m` változata vagy bitsliced AES) és ciklusos, kisebb LEA-kód mérése megmutatná, mennyi a különbség az algoritmusok és mennyi a megvalósítási stílus között.

**További platformok.** Más Cortex-M magokon (M0+, M33), valamint RISC-V és AVR mikrovezérlőkön végzett mérések megmutatnák, mennyire általánosíthatók az eredmények.

**Energia- és oldalcsatorna-mérés.** Precíziós árammérővel az algoritmusok tényleges energiaigénye, oszcilloszkóppal pedig a teljesítményanalízissel szembeni ellenállásuk is vizsgálható lenne.

**Azonos funkcionalitású összevetés.** Az AES és a LEA hitelesített üzemmódjainak (például GCM, CCM) mérése lehetővé tenné, hogy az Ascon-AEAD128-at azonos szolgáltatást nyújtó megoldásokkal hasonlítsuk össze, és ellenőrizné a CCM-re adott becslést.

# Irodalomjegyzék


[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Pearson, 2017.

[2] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*, Springer, 2010.

[3] C. E. Shannon, *Communication Theory of Secrecy Systems*, Bell System Technical Journal **28** (1949), no. 4, 656–715.

[4] NIST, *Advanced Encryption Standard (AES), FIPS PUB 197*, National Institute of Standards and Technology, 2001.

[5] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer, 2002.

[6] K. A. McKay, L. Bassham, M. S. Turan, and N. Mouha, *Report on Lightweight Cryptography*, NIST Internal Report 8114 (2017).

[7] A. Biryukov and L. Perrin, *State of the Art in Lightweight Symmetric Cryptography*, IACR ePrint Archive 2017/511 (2017).

[8] G. Hatzivasilis, K. Fysarakis, I. Papaefstathiou, and C. Manifavas, *A Review of Lightweight Block Ciphers*, Journal of Cryptographic Engineering **8** (2018), no. 2, 141–184.

[9] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, *Ascon v1.2: Lightweight Authenticated Encryption and Hashing*, Journal of Cryptology **34** (2021), no. 3. 33. cikk.

[10] NIST, *Lightweight Cryptography Standardization Process: NIST Selects Ascon*, National Institute of Standards and Technology, 2023.

[11] NIST, *Ascon-Based Lightweight Cryptography Standards for Constrained Devices (NIST SP 800-232)*, National Institute of Standards and Technology, 2025. Elérhető: `https://csrc.nist.gov/pubs/sp/800/232/final` (utolsó megtekintés: 2026. október 26.)

[12] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, *The SIMON and SPECK Families of Lightweight Block Ciphers*, IACR ePrint Archive 2013/404 (2013).

[13] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, *Notes on the Design and Analysis of SIMON and SPECK*, IACR ePrint Archive 2017/560 (2017).

[14] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, *SIMON and SPECK Implementation Guide, Version 1.1*, National Security Agency, 2019.

[15] D. Hong, J.-K. Lee, D.-C. Kim, D. Kwon, K. H. Ryu, and D.-G. Lee, *LEA: A 128-Bit Block Cipher for Fast Encryption on Common Processors*, WISA 2013, Lecture Notes in Computer Science **8267** (2013), 3–27.

[16] ISO/IEC, *Information security – Lightweight cryptography – Part 2: Block ciphers (ISO/IEC 29192-2:2019)*, International Organization for Standardization, 2019.

[17] D. Dinu, A. Biryukov, J. Großschädl, D. Khovratovich, Y. Le Corre, and L. Perrin, *Triathlon of Lightweight Block Ciphers for the Internet of Things*, NIST Lightweight Cryptography Workshop (2015).

[18] M. Cazorla, K. Marquet, and M. Minier, *Survey and Benchmark of Lightweight Block Ciphers for Wireless Sensor Networks*, SECRYPT 2013 (2013), 543–548.

[19] A. Bogdanov, D. Khovratovich, and C. Rechberger, *Biclique Cryptanalysis of the Full AES*, ASIACRYPT 2011, Lecture Notes in Computer Science **7073** (2011), 344–371.

[20] A. Bogdanov and V. Rijmen, *Linear Hulls with Correlation Zero and Linear Cryptanalysis of Block Ciphers*, Designs, Codes and Cryptography **70** (2014), no. 3, 369–383.

[21] L. Song, Z. Huang, and Q. Yang, *Automatic Differential Analysis of ARX Block Ciphers with Application to SPECK and LEA*, ACISP 2016, Lecture Notes in Computer Science **9723** (2016), 379–394.

[22] Y. Chen, Z. Bao, and H. Yu, *Differential-Linear Approximation Semi-Unconstrained Searching and Partition Tree : Application to LEA and Speck*, ASIACRYPT 2023 (IACR ePrint Archive 2023/1414) (2023).

[23] M. M. Hossain, M. Fotouhi, and R. Hasan, *Towards an Analysis of Security Issues, Challenges, and Open Problems in the Internet of Things*, 2015 IEEE World Congress on Services (2015), 21–28.

[24] Ericsson, *Ericsson Mobility Report June 2026*, Telefonaktiebolaget LM Ericsson, 2026. Elérhető: `https://www.ericsson.com/mobility-report` (utolsó megtekintés: 2026. október 26.)

[25] A.-R. Sadeghi, C. Wachsmann, and M. Waidner, *Security and Privacy Challenges in Industrial Internet of Things*, 52nd ACM/EDAC/IEEE Design Automation Conference (DAC) (2015), 1–6.

[26] E. Biham and A. Shamir, *Differential Cryptanalysis of DES-like Cryptosystems*, Journal of Cryptology **4** (1991), no. 1, 3–72.

[27] M. Matsui, *Linear Cryptanalysis Method for DES Cipher*, EUROCRYPT '93, Lecture Notes in Computer Science **765** (1993), 386–397.

[28] D. Khovratovich and I. Nikolić, *Rotational Cryptanalysis of ARX*, FSE 2010, Lecture Notes in Computer Science **6147** (2010), 333–346.

[29] P. C. Kocher, *Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*, CRYPTO '96, Lecture Notes in Computer Science **1109** (1996), 104–113.

[30] TTA, *128-bit Block Cipher LEA (TTAK.KO-12.0223)*, Telecommunications Technology Association, Korea. A KISA által terjesztett specifikáció és tesztvektorok. Elérhető: `https://seed.kisa.or.kr` (utolsó megtekintés: 2026. október 26.)

[31] KISA, *LEA C Standalone forráskód, v1.3 (LEA_C_Standalone_src.zip)*, Korea Internet & Security Agency. Elérhető: `https://seed.kisa.or.kr/kisa/Board/20/detailView.do`; változatlan tükör: `https://github.com/jinahya/kisa-lea` (utolsó megtekintés: 2026. október 26.)

[32] M. Dworkin, *Recommendation for Block Cipher Modes of Operation : Methods and Techniques (NIST SP 800-38A)*, National Institute of Standards and Technology, 2001.

[33] V. Rijmen, A. Bosselaers, and P. Barreto, *Optimised ANSI C code for the Rijndael cipher (now AES), rijndael-alg-fst.c, version 3.0*, a FreeBSD forráskódjában megőrzött változat, 2000. Elérhető: `https://github.com/freebsd/freebsd-src/blob/main/sys/crypto/rijndael/rijndael-alg-fst.c` (utolsó megtekintés: 2026. október 26.)

[34] D. J. Bernstein, *Cache-timing attacks on AES*, University of Illinois at Chicago, 2005.

[35] kokke, *tiny-AES-c : Small portable AES128/192/256 in C*, GitHub. Elérhető: `https://github.com/kokke/tiny-AES-c` (utolsó megtekintés: 2026. október 26.)

[36] C. Dobraunig and M. Schläffer, *ascon-c : Ascon reference and optimized implementations in C, crypto_aead/asconaead128/bi32*, GitHub, 2026. Commit 446347f. Elérhető: `https://github.com/ascon/ascon-c` (utolsó megtekintés: 2026. október 26.)

[37] NIST, *Submission Requirements and Evaluation Criteria for the Lightweight Cryptography Standardization Process*, National Institute of Standards and Technology, 2018. Elérhető: `https://csrc.nist.gov/projects/lightweight-cryptography` (utolsó megtekintés: 2026. október 26.)

[38] STMicroelectronics, *RM0394 Reference Manual : STM32L41xxx/42xxx/43xxx/44xxx/45xxx/46xxx Advanced Arm-based 32-bit MCUs*, STMicroelectronics. Elérhető: `https://www.st.com/en/microcontrollers-microprocessors/stm32l432kc.html` (utolsó megtekintés: 2026. október 26.)

[39] ARM, *Cortex-M4 Technical Reference Manual (ARM DDI 0439B)*, ARM Limited, 2010.

[40] Free Software Foundation, *Using the GNU Compiler Collection (GCC) : Options That Control Optimization*, Free Software Foundation. Elérhető: `https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html` (utolsó megtekintés: 2026. október 26.)

[41] P. Derbez, P.-A. Fouque, and J. Jean, *Improved Key Recovery Attacks on Reduced-Round AES in the Single-Key Setting*, EUROCRYPT 2013, Lecture Notes in Computer Science **7881** (2013), 371–387.

[42] T. Ashur and A. Luykx, *An Account of the ISO/IEC Standardization of the Simon and Speck Block Cipher Families*, Security of Ubiquitous Computing Systems, Springer (2021), 63–78.

[43] R. Rohit, K. Hu, S. Sarkar, and S. Sun, *Misuse-Free Key-Recovery and Distinguishing Attacks on 7-Round Ascon*, IACR Transactions on Symmetric Cryptology **2021** (2021), no. 1, 130–155.

[44] P. Schwabe and K. Stoffelen, *All the AES You Need on Cortex-M3 and M4*, SAC 2016, Lecture Notes in Computer Science **10532** (2017), 180–194.

[45] A. Moradi, A. Poschmann, S. Ling, C. Paar, and H. Wang, *Pushing the Limits: A Very Compact and a Threshold Implementation of AES*, EUROCRYPT 2011, Lecture Notes in Computer Science **6632** (2011).

[46] S. Peng, K. Hu, J. He, and M. Wang, *Improved Key Recovery Attacks of Ascon*, IACR ePrint Archive 2025/1029 (2025).

[47] ISO/IEC, *Information technology – Automatic identification and data capture techniques – Part 22: Crypto suite SPECK security services for air interface communications (ISO/IEC 29167-22:2026, 2. kiadás; első kiadás: 2018)*, International Organization for Standardization, 2026.

[48] ISO/IEC, *Information technology – Automatic identification and data capture techniques – Part 21: Crypto suite SIMON security services for air interface communications (ISO/IEC 29167-21:2026, 2. kiadás; első kiadás: 2018)*, International Organization for Standardization, 2026.

## Nyilatkozat

Alulírott Kárász Barna Kristóf mérnökinformatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Számítógépes Algoritmusok és Mesterséges Intelligencia Tanszékén készítettem, mérnökinformatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumában tárolja.

Szeged, 2026. október 26.

_____

Kárász Barna Kristóf

## Köszönetnyilvánítás

Köszönöm témavezetőmnek, Dr. Békési Józsefnek, hogy a témaválasztástól a dolgozat lezárásáig segítette a munkámat, és mindig hasznos észrevételekkel látott el.

Köszönöm a családomnak és a barátaimnak, hogy a tanulmányaim alatt végig mellettem álltak.

## Mellékletek

A dolgozat elektronikus melléklete egyetlen, legfeljebb 5 MB-os ZIP-fájl, amelyet a dolgozattal együtt a CooSpace-be töltöttem fel. A dolgozat PDF-változata mellett a mérések megismétléséhez szükséges összes fájlt tartalmazza.

**1. Forráskód** (STM32CubeIDE-projekt)

- `main.c` – önellenőrzés, mérési ciklus, statisztika, kiírás;
- `rijndael.c/h` (AES-128, a tervezők kódja), `speck.c/h` (NSA), `lea.c/h` (KISA), `ascon_aead128.c/h` (`ascon-c`) – az algoritmusok;
- `aes.c/h` – a `tiny-AES-c`, az összevető méréshez;
- `forras/` – a KISA LEA-csomagjának és az `ascon-c` könyvtárnak a felhasznált eredeti fájljai a licencekkel, az átvett kód ellenőrzéséhez.

**2. Mérési naplók**

- `meres-O0.csv` … `meres-Oz.csv` – a hat optimalizációs szint (`-O0`, `-O2`, `-O3`, `-Ofast`, `-Os`, `-Oz`) mérési sorai körönként;
- `osszesito_median.csv` – a mediánok és a származtatott értékek (CPB, feldolgozási sebesség, idő) szintenként;
- `memoria.csv` – a flash- és a veremigény szintenként.

**3. Kiértékelés**

- `graphics.py` – a naplók feldolgozása és az 5. fejezet ábráinak előállítása (Python, matplotlib);
- `memoria.py` – a kódméret és a veremigény kigyűjtése a `.map` és a `.ci` fájlokból;
- az ábrák PDF-változata.